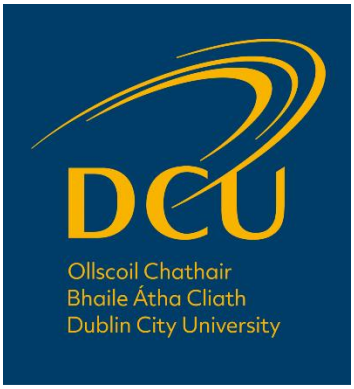


Final Year Project Report

Modelling the orbit and ideal launch windows to reach Jupiter and beyond.

Name: Orlaith Ní Dhúill

Student Number:

Class: PHA4

Date: Tuesday, 11th April 2023.

Supervisor: Dr. Samuel McKeague

Declaration

Name: Orlaith Ní Dhúill
Student ID Number:
Programme: PHA4
Module Code: PS451
Assignment Title: Modelling the orbit and ideal launch windows to reach Jupiter and beyond.

I understand that the University regards breaches of academic integrity and plagiarism as grave and serious.

I have read and understood the DCU Academic Integrity and Plagiarism Policy. I accept the penalties that may be imposed should I engage in practice or practices that breach this policy.

I have identified and included the source of all facts, ideas, opinions, viewpoints of others in the assignment references. Direct quotations, paraphrasing, discussion of ideas from books, journal articles, internet sources, module text, or any other source whatsoever are acknowledged and the sources cited are identified in the assignment references.

I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others save and to the extent that such work has been cited and acknowledged within the text of my work.

I have used the DCU library referencing guidelines (available at: http://www.library.dcu.ie/LibraryGuides/Citing&ReferencingGuide/player.html) and/or the appropriate referencing system recommended in the assignment guidelines and/or programme documentation.

By signing this form or by submitting this material online I confirm that this assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study.

By signing this form or by submitting material for assessment online I confirm that I have read and understood DCU Academic Integrity and Plagiarism Policy (available at: http://www.dcu.ie/registry/examinations/index.shtml).

Name:

Date:_

Abstract

Humans have had their heads titled to the stars, seeking of answers far before any other science was even created and yet we've only ever explored as far as the moon. Although there are currently plans to visit Mars, this project aims to expand our hopes to venture even further! Recent research has shown remarkable promise that Jupiter's moons Europa and Callisto may be habitable and contain life or possibly hold life in the future. In preparation for possible journeys beyond the Main Asteroid Belt that separates us from our giant cohorts within the Solar System, this project report details the work outline required to obtain results on the optimal transfer window for a launch from Earth to Jupiter at Closest Approach using Hohmann Transfer. This was completed by modelling a computerised orbital path of both planets relative to each other and to the Sun. The data of which was used to complete the aims of the project, calculating best trajectory, launch windows, and rocket parameters in preparation of this launch. The results found that this mission could be completed as early as 2034. Though the mission would require time, the only flaw of the Hohmann Transfer and high precision mathematical investment, it could potentially lead to further missions, possibly manned, and pave the way for the future of life as we know it.

Acknowledgements

I wish to express my sincerest gratitude to my supervisor Dr. Samuel McKeague for assisting with the project and helping me throughout - This project couldn't have been completed without him! Additionally, I would like to thank Dr. Paul Swift, who provided invaluable resources and useful information throughout.

A note of immense appreciation to all of my past science, physics, mathematics, astronomy, and laboratory professors during my university studies for going above and beyond throughout, despite the pandemic and difficulties that came with it.

I also wish to acknowledge my lovely girlfriend and friends for their patience with me and ongoing support during my most challenging year.

Lastly, I wish to acknowledge my wonderful parents and siblings for helping me along my university and educational journey and for never letting me give up on my astronomy dreams.

Table of Contents

Content Title…… Page Number

1. Preliminary

1.1 Declaration……..… i
1.2 Abstract & Acknowledgements…….…. ii
1.3 List of Figures……...…….…. iv

2. Introduction

2.1 Background History, Concepts & Theory…….…….. 1
2.2 Aims & Objectives…….……. 7

3. Data Analysis

3.1 Experimental Method(s)…….. 7
3.2 Modelling & Animation……... 8
3.3 Ideal Rocket Equation……. 17

4. Results

4.1 Data Results……. 18
4.2 Determination of Optimal Transfer Window…… 27
4.3 Effects of a Delayed Launch……. 28
4.4 Heavy Payload Delivery on a Hydrogen based Rocket…… 29

5. Discussion of Conclusions

5.1 Suitability of Optimal Transfer Window for Jupiter…… 31
5.2 Practicality of Heavy Payload Delivery……... 32
5.4 Conclusions & Summary of Results…..…… 32

6. **References** ……. 33

7. Appendices

8.1 Appendix 1 – Shell Script Used …… 36
8.2 Appendix 2 – Risk Assessment …….…… 47

List of Figures

Figure 1. JWT – Jupiter (Including ring, poles, and moons)……... 1
Figure 2. Voyager 1 – Great Red Spot and Gaseous Atmosphere……... 1
Figure 3. Juno (Artist Concept). …….. 2
Figure 4. Voyager Spacecraft (Artist Concept). …….. 2
Figure 5. Jupiter and Earth at Closest Approach (in Opposition).. ……. 5
Figure 6. Kepler's 1 st Law – Orbital. ……. 5
Figure 7. Kepler's 2 nd Law – Areas. …….. 6
Figure 8. Kepler's 3 rd Law – Harmonics.. ……. 6
Figure 9. Expectational Plan for Coded Orbital Model. …….. 9
Figure 10. Output JPL Horizons Data plot for launch date. …… 15
Figure 11. Screenshot of Example Critical Angle Measurement. ……... 15
Figure 12. Example '1' - Orbits of Planets and distance between them. …… 19
Figure 13. Example '2' - Orbits of Planets and Distance between them.…… 19
Figure 14. Example '3' - Orbits of Planets and Distance between them.…… 20
Figure 15. Total Days the code is run. Ex 1; 1 month, Ex 2; 6 months, Ex 3;1Y……... 20
Figure 16. Total Number of Days the code is run. Example 4; 5 Years…….. 20
Figure 17. Total Number of Days the code is run. Example 5; 25 Years…… 20
Figure 18. Date & Distances of all Opposition Occurrences (25 Years)……. 21
Figure 19. Screenshot of Top 5 Shortest Oppositions over next 25 Years…….. 21
Figure 20. Outputs for Determination of Closest Approach over the next 25 years…...…… 22
Figure 21. Date determination from exact date of Closest Approach……. 22
Figure 22. JPL Horizons Data; Date of Closest Approach…….. 23
Figure 23 Start of Trajectory for Hohmann Transfer at Closest Approach……. 24
Figure 24: Mid-point of Trajectory for Hohmann Transfer at Closest Approach……24
Figure 25: Late stage of Trajectory for Hohmann Transfer at Closest Approach……25
Figure 26: Completed Trajectory of Hohmann Transfer at Closest Approach……... 25
Figure 27: Plot Output – 'Mass - Velocity Relation' of Saturn V…….. 26
Figure 28. Plot Output – 'Mass Ratio – Velocity Relation' of Saturn V…… 27
Figure 29. Rocket Payload Comparison to Other Spacecraft, (Saturn V)…….. 27
Figure 30. Plotted output of trajectory for Hohmann Transfer of Delayed Launch……28
Figure 31. Zoomed-in Shot of the Missed Rendezvous Point for Closest Approach……. 29

2. Introduction

2.1 Background History, Concepts & Theory.

Background History

The next step in space exploration after venturing towards Mars[1], will hopefully be a trip towards the outer regions of the Solar System such as a mission to Jupiter. Jupiter is thought to have very severe and fluctuating temperatures, pressures, and materials making the planet's environment far too harsh to sustain life or for future human generations to adapt to. That said, a handful of Jupiter's 92 moons[2] may support life or could be habitable in the future. Included of which are three of Jupiter's largest moons, Europa, Callisto and Ganymede respectively. While Europa is approximately the same size as our Earth's moon, Ganymede, the largest moon in the Solar System, is larger than planet Mercury. These moons are thought to be habitable due to the evidence gathered over recent years that they have subsurface saltwater oceans[3]. Even if we may not plan to send manned missions to these moons it may be constructive to consider unmanned missions for research purposes. The frozen moon Europa, which conceals a large ocean, has been a main focus for researchers looking into whether life may reside elsewhere in the solar system. To that purpose, the Europa Clipper will go to the Jovian moon, with a launch date as early as 2024. The European Space Agency (ESA) will also launch the 'Jupiter Icy Moons Explorer' (JUICE)[4,5] in April 2023 to examine three of the Galilean moons; Ganymede, Callisto, and Europa.

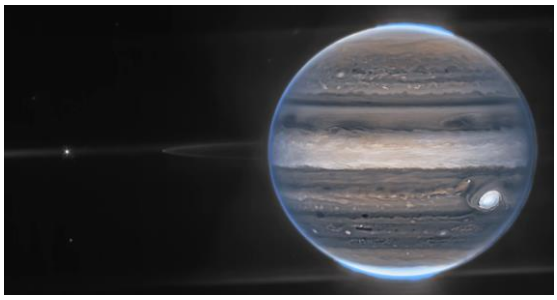


Figure 1.
JWT – Jupiter (Including ring, poles, and moons).



Figure 2.
Voyager 1 – Great Red Spot and Gaseous Atmosphere.

It has been known since Galileo's early astronomy that Jupiter is the largest planet in our solar system. Jupiter has long been known because it can be seen without the use of advanced telescopes. It's also the oldest and very first formed planet following the formation of the Sun, created less than three million years after, approximately 4.603 billion years ago. Jupiter was formed similarly to the Sun as it was born from the same cloud of molecular gas that gave birth to the Sun. However, it was formed into an orbital planet of the solar system because it did not have sufficient mass to fuse hydrogen into helium. Although it is not a star, Jupiter's atmosphere consists of very dense and relatively dry compounds of hydrogen, helium, and much smaller amounts of methane and ammonia.

This creates the appearance of 'swirling' gas clouds as seen on the outermost layer. Storms like the 'Great Red Spot' have been ongoing for hundreds of years. With a potential wind gust range of 435 km/h to 680 km/h and an estimated 16,000 km in diameter, the Great Red Spot is regarded to be the greatest storm in the solar system. Jupiter is a gas giant that has no solid surface; however, the planet possesses a solid inner core with a composition of hard heavy rock, iron, and frozen water. Jupiter also has rings, but unlike Saturn they are unfortunately far too faint to see by us here on Earth. In terms of Jupiter's position relative to the Sun, it is just over 5 times further away than Earth, approximately 740.83 million km in distance. As the planet is so massive, it rotates much more quickly than Earth and therefore Jupiter completes one full 360-degree rotation within only 9 hours and 56 minutes. Although it has much shorter days than Earth, Jupiter completes one full orbital rotation of the Sun or a 'Jupiter Year' in what would be equivalent to 11.8 years here on Earth. Jupiter has been visited or passed by several different spacecraft, orbiters, and probes, such as Pioneer 10 and 11, Voyager 1 and 2, Cassini, New Horizons, and Juno[6,7]. A planet with promise, Jupiter and particularly its habitable moons should be considered for future missions as technology advances into a new era of space exploration.



Figure 3. Juno (Artist Concept).

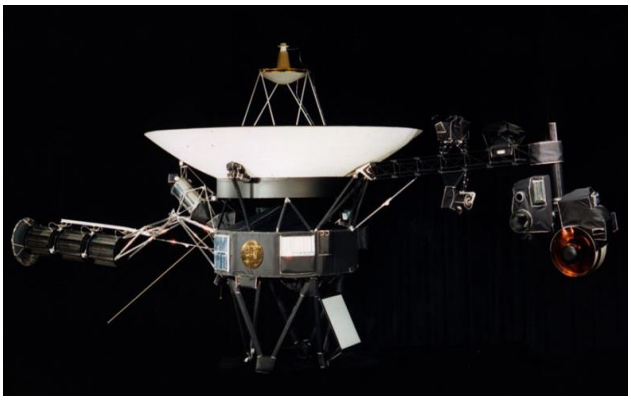


Figure 4. Voyager Spacecraft (Artist Concept).

Background Concepts

So, what would be the best approach to undergo such a long-distance mission? The answer lies with the orbital mechanics and advantages of Hohmann Transfer. This is an orbital manoeuvre used to transport a spacecraft between two circular orbits around a central body that are at different orbital positions relative to one another. There are many benefits to this manoeuvre, mainly that it utilises a single spacecraft rocket engine firing at a specific location in the lower orbit to conserve rocket fuel. The spacecraft's orbit shifts from a circular orbit to an oval orbit as a result of this firing, which gives the orbit more energy and drives it further away from the central body, then looping into the orbit of the outer body. A mission to Jupiter could be achieved with optimum efficiency using Hohmann Transfer, while assuming the orbital paths of both planets are aligned symmetrically in closest approach[8].

There are various step to take in order to calculate the Hohmann Transfer (HT). Firstly, we may identify our constants. The main physical constants to consider are the distances of both planets from the centre point of the Sun (*De* and *Dj*) and the standard gravitational parameter, which is just the gravitational constant times the mass of the Sun. The next step is to determine the orbital periods of the two planets (*Pe* and *Pj*) in seconds and the semi-major axis of the transfer orbit. The semi-major axis of an ellipse may be considered as the distance from its centre to its most distant side. This ellipse can then be used as a path the spacecraft will take from earth to Jupiter. The semi-major axis will be denoted by the variable α(HT) such that;

$$\alpha(HT) = \frac{(De + Dj)}{2} \quad (1)$$

Thereafter, the period of the orbit must be obtained. This may be done using Kepler's Third Law. This states that the squares the orbital periods of planets are directly proportional to the cubes of the semi-major axes of their orbits. This implies that the period for a planet to orbit the Sun increases rapidly with the radius of its orbit. Therefore, for the period of the transfer orbit, the variable 'α' will be α(HT) such that;

$$P(HT) = \sqrt{\left(\frac{4\pi^2 \cdot \alpha^3}{GM}\right)} \quad (2)$$

Following this the velocity of the planets orbits around the Sun must be found in order to determine what adjustments must be made to the velocity of the spacecraft performing the manoeuvre. This will allow the spacecraft to enter the elliptical orbit from Earth to Jupiter. Where the velocity of Earth's and Jupiter's orbit is *Ve* and *Vj* respectively, then;

$$v(e) = \frac{(2\pi \times De)}{Pe} \quad (3)$$

$$v(j) = \frac{(2\pi \times Dj)}{Pj} \quad (4)$$

Once the velocity of both circular orbitals is obtained, the velocity of the elliptical transfer orbit at its Perihelion. This is the end of the ellipse which is closest to the Sun. Due to the elliptical transfer orbit being closer to the Sun at the end with Earth's orbit than that of Jupiter's. The velocity of the orbit at the perihelion will assist in determining how best to launch the spacecraft into the elliptical orbit from Earth's orbit into Jupiter.

$$v(Perihelion) = \frac{2\pi \times a(transfer)}{P(transfer)} \times \left(\sqrt{\left(\frac{2 \times a(transfer)}{De}\right)} - 1\right) \quad (5)$$

Following the velocity of the elliptical orbit at its perihelion, the next step would be to calculate the change in velocity the spacecraft would require changing the orbit of the spacecraft from Earth's orbit into the transfer orbit to Jupiter. The rocket will require a small burst of energy to start on the elliptical transfer orbit. This change in velocity will be helpful in determining the fuel required by the spacecraft. This change in velocity may be calculated by;

$$\Delta v(e) \; = \; v(Perihelion) \; - \; v(e) \qquad (6)$$

The last few steps involve finding the velocity of the elliptical orbit at its aphelion of the elliptical orbit. The aphelion is the end of the ellipse furthest from the Sun, which may also be described as the end of the ellipse that aligns with the orbit of Jupiter.

$$v(Aphelion) = \frac{2\pi \times a(HT))}{P(HT)} \times \left(\sqrt{\left(\frac{2 \times a(HT)}{Dj} \right)} \; - \; 1 \right) \qquad (7)$$

Next the change in velocity necessary to send the rocket from an elliptical transfer into Jupiter's orbit must be obtained. The burst of velocity required should be equivalent to the difference in velocity of Jupiter's orbit and the velocity of the elliptical orbit at its aphelion. This velocity may be calculated by;

$$\Delta v(j) \; = \; v(j) \; - \; v(Aphelion) \qquad (8)$$

Lastly, the time of flight (TOF) should be achieved. This is the amount of time required for the rocket or spacecraft to journey from Earth to Jupiter. It is equivalent to half the period of the transfer orbit. At first, the value will be calculated in seconds this can then be converted to days and years. This value is of extreme importance to the overall launch as it will determine the launch date required to meet the small window of closest approach and will be useful for fuel efficiency.

$$TOF \; = \; \frac{1}{2} \times P(HT) \qquad (9)$$

Focussing on the concept of 'Closest Approach'. This refers to the shortest distance between two planets centres when they are externally tangent to one another. Jupiter just recently experienced its closest approach relative to Earth since October of 1963. This occurred on September 26th 2022, just before Jupiter's most recent opposition from the Sun on the 26th. Jupiter's opposition occurs every 13 months or so which doesn't make it a rare phenomenon, unlike its closest approach to Earth which only occurs once every 59 years or so. However, on this occasion the timing was perfect for both an opposition and a closest approach to work in harmony providing a distance of only 590 million kms between Jupiter and Earth. This is a mere 60% the distance of the planet from us at its farthest, at approximately 960 million kms away. The closest approach allowed astronomers with all levels of equipment to see the planet with more detail, even simple binoculars allowed a view of the planet's larger moons and the detail of the planet itself. For the naked eye the planet was easily recognisable being the largest and brightest object in the night sky, second only to the moon! Although we may use the simple 'every 59 years' rule-of-thumb

to predict when the approximate next closest approach of the two planets may occur it's also very easy to use Kepler's Laws[9] and Newton's Laws of Motion to calculate this. Where the Sun is the centre point of a 2D cartesian planar surface, the two planets rotate in an almost circular orbit. This can be animated, and the associated data may be sought detailing the next closest approach.

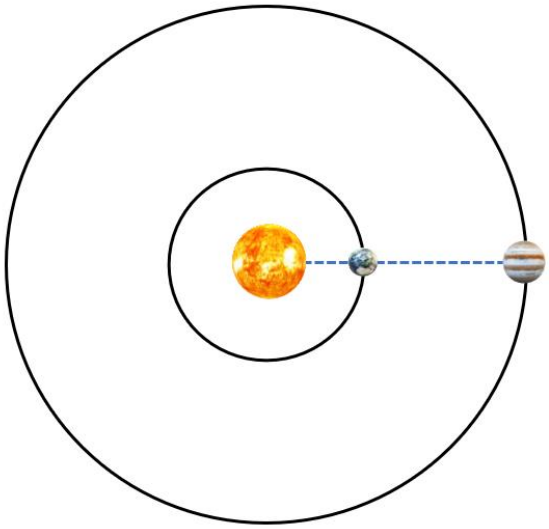


Figure 5. Illustration of Jupiter and Earth at Closest Approach (in Opposition).

Kepler's Laws of Planetary Motion express the orbits of planets around the Sun. As expected by the title, these laws were produced by astronomer Johannes Kepler in the years 1609 to 1619. There are three laws;

1. Orbital Law;

   All planets have elliptical orbits around the Sun, where the Sun is at one of the two foci.

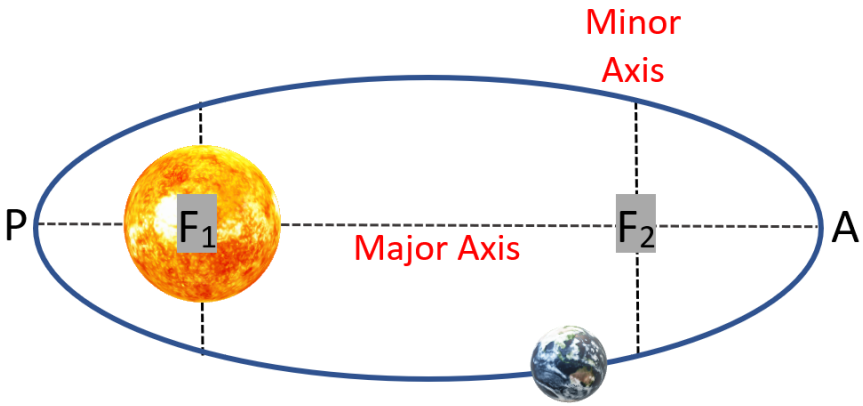


Figure 6. Illustration of Kepler's 1st Law – Orbital.

2. Law of Areas;

   A radius vector joining any planet to the Sun will sweep out equal areas within equivalent time lengths.

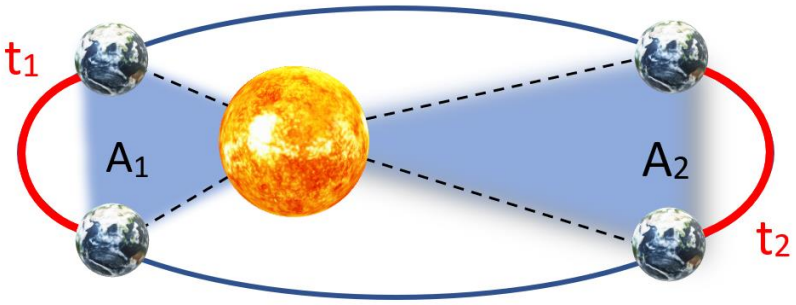


Figure 7. Illustration of Kepler's 2nd Law – Areas.

3. Periodic Law of Harmonics;

   The square of a planet's orbital time it directly proportional to its average distance from the Sun cubed.

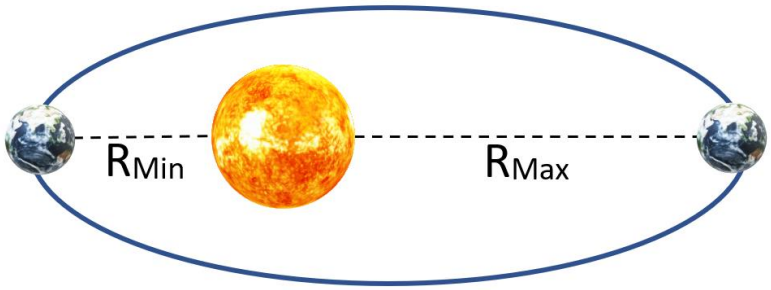


Figure 8. Illustration of Kepler's 3rd Law – Harmonics..

Kepler made these mathematical and revolutionary assumptions of planetary motion by abandoning both uniform and circular motion in his observations. He noticed that the solar planets all rotated about the Sun with elliptical paths, with non-uniform velocities.

2.2 Aims & Objectives

The aims and objectives hoped to be achieved upon completion of this project are noted here;

1. The modelling of relative positions of Earth and Jupiter in their orbit to the sun over a specific period of 25 years (2022-2047).

2. A computer code calculation to estimate the optimal transfer windows and orbits, inclusive of time and velocity changes required, for a successful Hohmann Transfer.

3. Investigation of effects if launch is delayed from ideal launch window.

4. Calculation of the energy required by a gravitational assist from ideal launch window to obtain escape velocity from the Sun.

5. Logical determination on whether it is realistic to deliver a payload of more than 20t into an orbital transfer with a liquid hydrogen-based rocket.

6. If so/not, a determination on realistic payload size to send to Jupiter and beyond.

3. Data Analysis

3.1 Experimental Method(s)

Naturally, as this is a computer-based project the 'Experimental Methods' are not all that complicated in terms of apparatus and required equipment. This project required only a computer, a strong internet connection and use of a Jupyter Notebook, which I accessed through use of Google Colab for convenience. Firstly, a 'Project Plan' was created between my tutor; Dr Mc Keague, and I. Whereby, deadlines and how to best approach the project aims in the most efficient order were discussed. The steps taken to undergo this project are outlined below in order of personal completion.

1. The core code to be created; would be that of the orbital model for both planets. This code would efficiently display the orbital velocities of both planets relative one another. This code would provide the data required to achieve an animation of the two planets orbiting the sun. It would also be interactively dependant on a timespan input which would then show the total orbital period being calculated on. Lastly, this code would contain the necessary data to provide the date of closest approach for interplanetary transfer using a Hohmann Transfer.

2. Once an estimation date was achieved for closest approach, then a Hohmann transfer[12] focussed code may be created. This code would display the trajectory of the transfer and assist in determination of the optimal expected launch date of the rocket from Earth.

3. The Hohmann Transfer code script may then be used to calculate the effects of a delayed launch on the overall success of the launch mission.

4. Thereafter, a code would be required to investigate the practicality of launching a heavy payload along the Hohmann transfer trajectory proposed in '2'.

5. On completion, the code and steps regarding the Ideal Rocket Equation would be created and assessed.

6. Lastly, the results of all the investigated code shells and data collected would eventually be used to complete data analysis, provide results and finally discussion on project completion.

7. Once the above steps are completed, and the correct observations are made, the report, slideshow and presentation poster may be created.

3.2 Modelling & Animation.

The modelling and animation of this investigation was divided into two different stages and code. Firstly, the modelling of the orbital data was needed in order to determine a date of closest approach on which a Hohmann Transfer would be used for launch. Secondly, the time of closest approach would be used to create a trajectory code for a model of the optimal Hohmann Transfer available to get to Jupiter from Earth. The breakdown of this code modelling work is described below, respective of each stage.

Orbital Data & Closest Approach Estimation - Part 1. Modelling.

This segment of the project required the bulk of the work and effort in terms of research and coding. Here, I had to first generate the orbital data required by both planets. This required a lot of physics prep work which I completed first on paper.

The main physical law to consider was Newton's Law of universal gravitation[15]. This law allows to calculate the force between two different objects in space. To calculate this '*Equation 10*' is used.

$$F = G\frac{m_1 m_2}{r^2} \qquad (10)$$

Whereby 'm_1' and 'm_2' are the mass of object 1 and object 2, respectively. 'r' is the distance between the centers of the masses of both object and G is the gravitational constant. In order to find the force exerted on the Earth and on Jupiter by the Sun we just plug in the values as such;

$$F_E = G\frac{(1.989_{\times 10^{30}}\ kg)(5.972_{\times 10^{24}}\ kg)}{1^2}$$

$$F_J = G\frac{(1.989_{\times 10^{30}}\ kg)(1.898_{\times 10^{27}}\ kg)}{5.2^2}$$

Using this law and Newton's Second Law of Motion[16], which states that the acceleration of any object is dependent on two core variables: the net force acting on the object and the mass of that object. The acceleration of an object, therefore, is proportional to the net force acting on the object, and inversely on the mass of the object. Therefore, as the force acting upon the object is increased, the acceleration of that object is also increased. Likewise, as the mass of an object is increased, the acceleration of that object is decreased. This equation may be simply put as follows;

$$F_{net} = m(a) \tag{11}$$

$$a = \frac{F_{net}}{m} \tag{12}$$

From '*Equation 12*', the acceleration of the planets may be calculated by;

$$a = G\frac{M}{r^2} \tag{13}$$

This calculation of acceleration is related only to the mass of the Sun and the distance between the planets and the centroid mass of the Sun. With a known value for acceleration, the velocity of the planets within a delta time value, denoted by 'dt', may be found by;

$$dv = a \times dt \tag{14}$$

From '*Equation 14*', the change of displacement may be derived by;

$$dd = (v + dv) \times dt \tag{15}$$

These equations must all be considered when creating a orbital, data concise model of the planets whereby the Sun is considered the centroid point in a vector space. The vector space considered here is a simple two-dimensional plane, whereby Earth and Jupiter are orbiting the centroid in distances of astronomical units (AU).

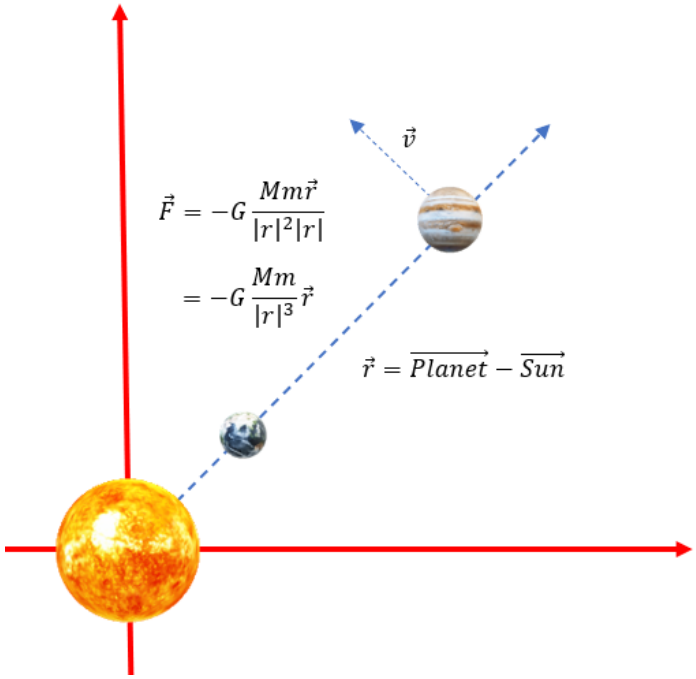


Figure 9. Illustration of Expectational Plan for Coded Orbital Model.

It may be noted that there is a minus symbol within the gravitational equation shown in 'Figure 9', this is seen here because the force direction is opposite the distance vector '$\vec{r}$'. The planets orbit in circular motion around the Sun due to centrifugal force, which is created due to the gravitational force acting upon them. One may think that this would cause the planets to fall inward in their orbits towards the Sun. However, the forward velocity of these planets, which is perpendicular to the Sun's gravitational force, in fact inversely pulls the planets away from the falling motion. The planets move rotationally with sufficient velocity to escape the gravitational pull caused by the Sun.

The full code used may be found in '*Appendix 1 – Shell Script Used – 1. Manual Data - Closest Approach*', however, for experimental method purposes, an in-depth description of individual code line is documented here in order of understanding. Bearing the astrophysics of the orbits in mind, the model was created in small steps suitable to the equations and constants. Firstly, I defined a table physical constants of the required values to be used within the solar orbital model;

Constant Header	Code Input Header	Value
Gravitational Constant	G	6.67e-11
Mass of Sun (kg)	Ms	1.989e30
Mass of Earth (kg)	Me	5.972e24
Mass of Jupiter (kg)	Mj	1.898e27
Astronomical Unit in (m)	AU	1.5e11
Hours*mins*secs	daysec	24*60*60
time difference	dt	1*daysec
Initial time	t	0.0
Earth velocity (m/s**2)	e_ap_v	29785
Jupiter velocity (m/s**2)	j_ap_v	13060
Gravitational Const on Earth	gconst_e	G*Me*Ms
Gravitational Const on Jupiter	gconst_j	G*Mj*Ms

Table 1. Table of Physical Constants (Replicated into code – 'Appendix 1')

Once the table of constants and values are created and defined into the code, the velocities and gravitational constants of Earth and Jupiter may be calculated using simple code based off the equations considered above. From there, the arbitrary model may be created and plotted using basic coordinate geometry. Using a simple x, y plane, we may determine the starting conditions of the model as both planets beginning on the x-coordinate at their relative distances to the Sun in AU. As seen in the code, the x-coordinate of the planets are both defined in AU, whereby Earth is naturally 1*AU in distance from the centroid position of the Sun, and likewise Jupiter sits at a position of 5.2* AU's away from the Sun. These are suitable locations as this is merely an average based model. Of course, the Sun remains plotted at a 0,0 point to remain at central position of the axes. Once, the starting conditions of the plotted model are setup as stationary points, the model must then account for the time difference 'dt' and the positional data is saved to a 'list', marking each positional coordinate of the planets and of the Sun. The saved lists will then be used in observation and analysis.

We can then simulate and animate the model of the orbits. This is where the intended timespan being observed will be input. This is done using a 'while' loop, whereby the simulation will continue until the timespan is reached, achieved by the code line; 'while t<1*30*daysec:' for example. Focusing first on the planetary bodies, simulation conditions were set up for Earth. This planet provides a good starting point as it simulated most of the orbital data required, which can later be replicated with small adjustments made for Jupiter's path. This includes the radial positional vector of Earth from the Sun, the gravitational force acting on Earth in its rotational orbit, the velocity vector of Earth as it changes in position and lastly, the positional 'x', 'y' coordinate data of the planet. These calculations for modelling all required the physical constants to be defined first as shown in 'Table 1'.

Although this model only includes a simple 'x', 'y' plane, we know of course, that in reality, the planetary movements aren't restricted to a two-dimensional plane as they travel through space and time. Here, we are assuming the law of gravitation in a vector space. Therefore, '*Equation 10*' must be modified whereby the 'r' vector divisor is made 'r^3' rather than 'r^2'. To calculate the radial positional vector 'r'; the input coordinate positions of Earth were subtracted from that of the Sun. To find the cube of '$r\ (|r|^3)$', as modified in the equation, we may simply add both squared values of 'r_x' and 'r_y' and set it to the power of 1.5; 'cube_of_r_e = (rx**2+ry**2)**1.5'. Using this value as expected, by the final gravitation equation shown in 'Figure 8', we may then calculate the gravitational force on Earth in both the 'x' and 'y' directions using code lines 'fx_e = -gc_e*rx/cube_of_r_e' and 'fy_e = -gc_e*ry/cube_of_r_e' respectively. Once calculated, the velocity vector may be constructed using Newton's Laws of Motion as denoted by '*Equations 11*' and '*12*'. Here, this 'acceleration' was again calculated in the 'x' and 'y' directions; 'xve += fx_e*dt/Me' and 'yve += fy_e*dt/Me'. It may be noted that these velocity vectors are calculated after the time difference, denoted by 'dt'. The final calculation required by the model orbit was the positional displacement of the Earth relative to its starting coordinate position. This was calculated by multiplying the velocity, previously defined, by the time difference. This then denotes the distance travelled by the planet within the given timeframe. Lastly, to save these updated positional values to the; 'xelist.append(xe)', code line is used for both 'x' and 'y' positional values.

As aforementioned, this code can then be replicated and adjusted to Jupiter's orbital data. The adjustment required was to simply change the 'e' title within the code commands which denote Earth, to 'j', which obviously denotes an abbreviation for Jupiter. The constants pertaining to Jupiter were all created when defining the physical constants as detailed in '*Table 1'*. These constants would mandatorily be defined first, though the order of the planetary modelling is not relevant in this case. It would have the same effect to have created Jupiter's orbit before Earth's, it would by no means impact the completed model created. Lastly, the Sun's stationary position on the plane had to be created and inputted into the Matplotlib data. This was done similarly to the other planets using a velocity vector of zero and an unchanged positional data. Again, the only change in code was to use an abbreviation 's' instead of the 'e' or 'j' title. This of course, denoting the Sun's informational data as above. From there, the system had to take a time increment of increasing 'dt', this allowed it to move forward in time as we'd expect from the moment of the planets; 't +=dt'

All of the above code may be used for to determine a time of closest approach assuming the system is run from the last point of closest approach which in this case we may assume to be 26th September 2022, where the planets are only separated by a distance of 590.63km. We may assume this distance to be a starting point of 4.5AU from Earth for the purpose of creating a closest approach simulation.

Orbital Data & Closest Approach Estimation - Part 2. Animation.

Using the setup of the gravitational force acting upon the planets, the velocity vector of the planets in orbit and the positional data of the planets around the centroid point of the Sun, the animation element of the visual data may be created. The planets and the Sun are represented by markers and different colours suitable to their physical attributes to represent the data automation associated. For example, the Sun is shown by a stationary yellow and orange outlined marker set in the centroid 0, 0 position. While the planets are displayed using size appropriate green and blue, for Earth, and red and brown markers for Jupiter. It was also important to display the paths taken by the planets in their orbit. The 'line' function was used to do this, also with appropriate colours chosen to display the path travelled, the linewidth of which was set at '3' for best visual results. To allow the animation to display the approximate distance between the two planets, it was essential to connect a distance vector between them. To display this, the 'arrow' function was used, with an accompanying text box displaying its distance length in kms. This is all available through Matplotlib animation functions.

The first element of the animation process was to set-up the axes parameters of the plane this is done by first determining a suitable figure size for the observation; 'fig, ax = plt.subplots(figsize=(6.5,6.5))'. By setting the aspect of the figure to 'auto' or automatic, the figure aspect size created will be suited to the data supplied by the code used for modelling. The visual colour may be adjusted suited to the program simulation. Of course, the obvious choice was originally black, as this could represent the darkness of the night sky, however, setting the 'facecolor' to 'w' or white created more visually clear results for the observation. Lastly, the axis elements are removed using the plt.axis('off'), this allows the simulation to run clearly as if in real astronomical observation. Lastly, an 'update loop', was used to display all the animated data, along with the planetary data previously set. When put together, it allowed the code to run the data in animated form to display the numerical data previously defined. To view the animation created, the IPython.display HTML function was used. This allowed the animation to be saved and used for analysis.

Hohmann Transfer - Part 1. Modelling.

The second stage in this investigation is that of the Hohmann Transfer trajectory. The full shell script code designed here can be found in *Appendix 1 – Shell Script Used – 3. Hohmann Transfer Trajectory'*. We know based on '*Equation 9'*, that the period or time of flight can be calculated with simplicity for a Hohmann transfer. After calculation, and verification from online resources '*14*' and '*16*' in '*References*', it is estimated that at closest approach a Hohmann Transfer would take an approximate periodic flight time of 2.729056 Years, which is equivalent to 997.8 days in flight time. Using NASA's JPL Horizons online Solar System database[17], I also worked a similar model of code as to that in the afore discussed '*Orbital Data & Closest Approach Estimation'***.** This database and ephemeris computation service allows anyone to access key Solar System data of highly accurate ephemerides for all solar bodies, including that of Earth and Jupiter. Obviously,

this database had to be installed and attached to the system to be used for code. This was done using 'pip install astroquery', which gained access to 'import jplhorizons's pre-calculated data of the planet's positions based on input time. The full shell code may be found in '*Appendix 1 – Shell Script Used – 2. JPL Horizons Data'.* This code allows the user to input a start date and simulation duration for which the model will run in animation. After it is run, the code produces an output of two different plots one animated to simulated the orbits of the planets and the other showing the exact starting positions of the planets relative to the centroid of the Sun in the 'x,y' plane. The '*JPL Horizons Data*' plot output may be used for comparison to the output of the '*Manual Data*' code and plots produced**.** For example, the '*Manual Data*' results provide a date of closest approach as _, when this date is input to the '*JPL Horizons Data*' code, the plot output of starting conditions, supports this estimation of closest approach as both codes provide a plot where both the planets are in planetary opposition and a point of relatively close proximity in their orbits. Although, the 'Manual Data' estimation of closest approach may have been sufficient for proceeding onto the next stage of creating the 'Hohmann Transfer' code of trajectory, it was helpful for calculation and certainty verification to also use the output of the data of the JPL Horizons Data.

The use of the plots for closest approach was invaluable for the creation of code for the Hohmann Transfer. Obviously the Hohmann Transfer requires the acceleration of the spacecraft onto an elliptical orbit meeting the orbit of Jupiter at the exact correct point of rendezvous. Using the distance the spacecraft would travel to complete an optimal Hohmann Transfer and the orbital period of Jupiter, 4332.6 days, it may be calculated that in order for the transfer to be completed at point of closest approach the spacecraft would be launched 997.8 days before the date of closest approach. Obviously, this is entirely dependent on the mass, velocity and thrust stage features of the spacecraft in question, however this is the generalised case. To launch the spacecraft at that optimal time; 997.8 prior to closest approach, It was simple to count backwards from the date of closest approach, showing that the idealised date of launch would be Thursday 8th of January 2034. This date was also verified by using the online resource[19] in references. By inputting this launch date into the '*JPL Horizons Data',* a plot of the planets real-time orbital positions relative to each other is produced for that date, see '*Figure 10*' below. From this outputted plot, we may measure the approximate phase or critical angle by which Jupiter would be ahead in its orbit. This angle was easily measured using an image measurement tool, which allows the user to calculate the angle between two points accurately with a protractor tool. This critical angle was measured to be 0.68°. This approximation angle was verified using '*Equations '16*' and '*17*' below. Once verified using equations, this critical angle will be input into the code to show the trajectory of the Hohmann Transfer as its completed.

$$\phi = 180° - \frac{1}{2} \cdot T_{Hoh} \cdot \theta_J \qquad (16)$$

$$T_{Hoh} = 2\pi \cdot \sqrt{\frac{\alpha_{Hoh^3}}{\mu}} \qquad (17)$$

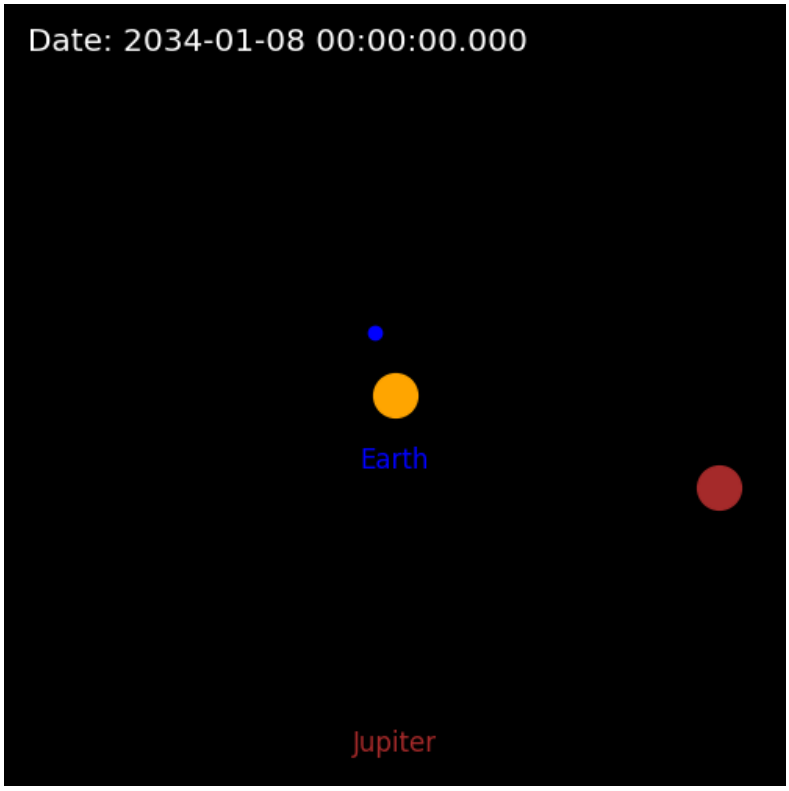


Figure 10. Output JPL Horizons Data plot for launch date.

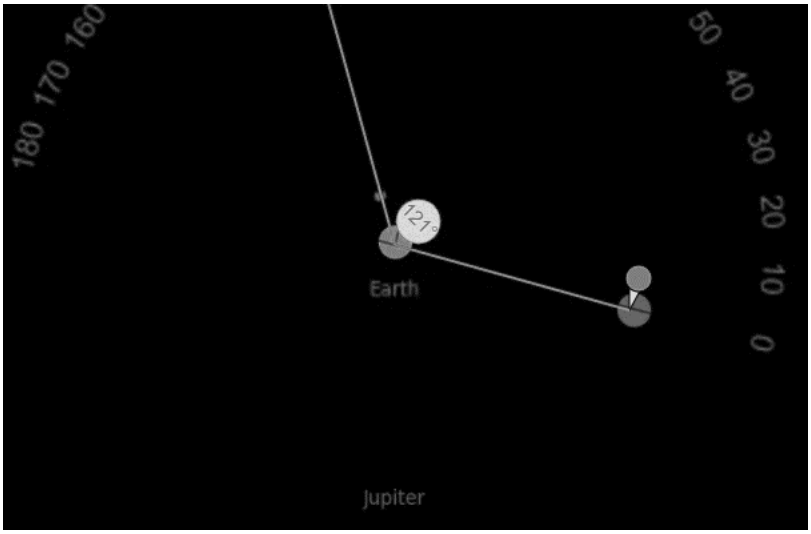


Figure 11. Screenshot of Example Critical Angle Measurement.

Focussing on the code itself, '*Matplotlib*' and '*Numpy*' were the predominant packages used for calculation of the Hohmann Transfer and 'animation' was imported to simulate the transfer. Similar to the '*Manual Data'* code, the orbits of both planets were first created. Obviously, this element of code is not focussed on a closest approach output but instead trajectory plotting. In order to do this; the plane is considered arbitrary and the

code creates an entirely new 'x' , 'y' plane focussed solely on the Hohmann Transfer trajectory. With this in mind, it was best to keep the code simple and focussed on the task at hand. Therefore, the '*matplotlib – circle( )*' function was used to display the orbits of the different planets where the Sun is the centroid point 0, 0 just as it was previously. The radius of the circle was defined in AUs.; 1 for Earth and 5.2 for Jupiter. With this function, it is also available to define the planetary colours, . The featured components of the planets, the Sun and the spacecraft could also easily be defined using the '*matplotlib – patch ( )'* function where we can define the planets as circles, all with respective colours and sizes associated with their true physical form. Following the physical creation of the planets and their orbits, two new functions had to be designed. The first, created a base frame suited to animate the data, this would include the initial starting positions of the planets and the associated rocket spacecraft that would perform the transfer. Using this initial positional data, the animation allows the manoeuvre trajectory to be simulated relative to the orbital time of the planets as they advance towards their closest approach.

To animate the orbital paths of the planets, the 'x' and 'y' coordinates are updated using the '*define(i)*' function. This function creates a loop similar to that of the '*Manual Data*' code previously used however, the code here is simplified to create the orbit using only basic geometry and '*Equations* '*18*' and '*19*' below for the updated 'x' and 'y' positions respectively. These coordinates are then updated using the loop as the code is run and progressed in the time of loop '*i*'. Obviously, we know the orbital period of Earth is 365.2 days and the orbital period of Jupiter is 4333 days. Therefore, it is very simple calculation of orbital position when compared to the earlier script.

$$sin\left(\frac{2(\pi)}{Orbital\ Period}\right) \times (i) \qquad (18)$$

$$cos\left(\frac{2(\pi)}{Orbital\ Period}\right) \times (i) \qquad (19)$$

Seeing as Jupiter is the destination of the transfer, some slight alterations were made to the code suited to the transfer. Since this Hohmann transfer requires the spacecraft to travel half the orbit, which is assumed circular here, the critical angle is multiplied by pi and divided by 180°. See '*Equation 20*', which is used to code Jupiter's path;

$$\sin or \cos\left[\left(\frac{2(\pi)}{Orbital\ Period}\right) \times (i) + \left(\frac{\pi(\alpha)}{180°}\right)\right] \qquad (20)$$

Lastly, the Hohmann transfer for the spacecraft's trajectory was created. This calculation required knowledge of the period of the transfer, the transfer orbit semi-major axis and most importantly, the critical angle that Jupiter is ahead by at launch date. This angle is of utmost importance, as if it is off by even the smallest angle it could widely impact the trajectory of the transfer and therefore miss the rendezvous requiring course correction. Just like in the case of the planets, the spacecrafts trajectory had to be defined using the updated 'x' and 'y' coordinates. This was done using '*Equation 21*';

$$\frac{(Period\ of\ Transfer) \times (1 - Critical\ Angle)}{(1+\cos\left[\left(\frac{2(\pi)}{Orbital\ Period}\right) \times (i)\right] \times \sin\left[\left(\frac{2(\pi)}{Orbital\ Period}\right) \times (i)\right])} \qquad (21)$$

Hohmann Transfer – Part 2. Animation.

The animation process of the Hohmann Transfer proved to be tricky. As with in the previous animation process, the '*matplotlib – animation*' feature was used here to simulate not only the orbits of the planets, but the trajectory and flight of the spacecraft directly from the Earth's orbit into Jupiter's orbit. Using the '*writer ( )*' function, the frames per second and bitrate of the simulation were controlled, this allowed the trajectory to be visually visible and much easier to follow. Thereafter, a plot layout was set whereby the font, text size and axes were set. Once completed, the trajectory path was set, this was done to show the different points in which the spacecraft would travel across in order to perform the Hohmann Transfer. This was done by creating a path of points using 'x' and 'y' coordinates, the period of the transfer and different periodic points in the time of the transfer. See '*Equation 22'*, which was used for each point. This was done by defining each point by a letter and increasing the time period for each 'x', 'y' defined coordinates. Lastly, the code is simulated using the '*animation.FuncAnimation*' function. This requires setting a frame and an interval number.

$$T.Orbit \text{ of S. Major Axis} \times \left(\frac{1-Transfer\ Orbit\ Eccentricity}{1+Transfer\ Orbit\ Eccentricity \times \frac{cos}{sin}\left(\frac{2\pi}{Period \times Time\ Increment}\right)}\right) \qquad (22)$$

3.4. Ideal Rocket Equation

The Classical Rocket Equation[20,21] describes the motion of a rocket. A rocket uses jet propulsion from energy provided by a propellant such as liquid hydrogen. This accelerates the rocket through its surrounding space and vacuum using thrust by expelling some of its mass with a high velocity. This allows the rocket to move due to the conservation of momentum. The maximum change of velocity without external force, such as drag, acting on the body may be calculated by '*Equation 23*';

$$\begin{aligned} \Delta v &= v_e \, ln \frac{m_i}{m_f} \\ &= v_f - v_i \\ &= I_{sp} \, g_0 \ln \frac{m_i}{m_f} \end{aligned} \qquad (23)$$

Whereby $'v_e'$ and '$I_{sp} \, g_0$' is the effective exhaust velocity; 'I_{sp}' is specific impulse in dimension of time, 'g_0' is standard gravity, and 'm_i', 'm_f' are the initial and final total mass of the rocket, respectively. The mass changes due to the rocket being fully fueled on launch ("wet mass") and without additional fuel mass on mission completion ("dry mass"). This equation can be used for many purposes, such as; change of velocity, a rocket's required mass ratio as a function of effective exhaust velocity ratio, or required fuel mass for a change in velocity.

Code Testing - Ideal Rocket Equation.

This code didn't require any modelling or animation therefore the only Python modules imported were; '*matplotlib.pyplot*' and '*math*'. The coded mass and velocity parameters were defined and based off of the most powerful hydrogen space rocket to date, Saturn V[22]. This rocket type was used for the Apollo missions to the Moon, and therefore would be the next suitable choice after SpaceX's Starship rocket[23], which relies on sub-cooled liquid methane as its propellant. Saturn V is estimated to have an exhaust velocity of 2.4 km/s, an initial mass at launch of 1.19e5 kg, and a dry mass when empty on rocket fuel of approximately 1e4kg. These are the essential requirements for input of the code;

Firstly, the constants were defined, such as the exhaust velocity at 2400km/s, the masses. Thereafter, arrays were created for the derivatives of this equation; inclusive of mass ratio and change of velocity. From there, a '*while loop*' was created to append the arrays and work them into the equation. Lastly, the data created was used to plot the 'Mass – Velocity' relation and the 'Mass Ratio – Velocity Change' relation. These plots would be useful in determination of fuel ratio, which typically amounts to approximately 77% of the overall mass of the rocket.

4.Results

4.1 Data Results

The results of the project investigation are shown here in order of completion. The investigation focussed mostly on obtaining the date of closest approach and a Hohmann transfer trajectory from Earth to Jupiter. Both aims were achieved and are examined below.

Closest Approach Determination

'Figures '*12*', '*13*' and '*14*' display screenshots of the plot output shown for different points in the orbits of the planets determined by the code in '*Appendix 1 – Shell Script Used – 1. Manual Data - Closest Approach*'. These outputs are examples of the observational results which display the mathematics and physics within the body of the code. These screenshots from the simulation display how the planets orbit the Sun due to its substantial gravitational pull. When the code is run, they are accompanied by a data output of the approximate distance that lies between the planets brought by the distance vector created between them. This date of closest approach was determined, based on to the output of days since the last date of closest approach. Assuming the code is run from the date of last closest approach, September 26th 2022, the code produces the day count following that date when there is a minimum distance between the Earth and Jupiter for the duration the code is run. The code is also programmed to output the total number of days that the observation is run. This can be seen in '*Figures '14', '15'* and *'16'*. It was initially intended, that the code print a table of all of 23 occurrences of opposition between the Earth and Jupiter which occur every 13 months. This is shown in '*Figure 17*'. However, this method proved problematic and required unnecessary data examination, therefore for convenience the code was altered to output only the date of opposition with the absolute shortest distance in kilometres.

'*Figure 19'*, shows a screenshot of the estimated date output of the closest approach or minimum opposition in the form of the time since the last closest approach and the distance between the planets on that day.

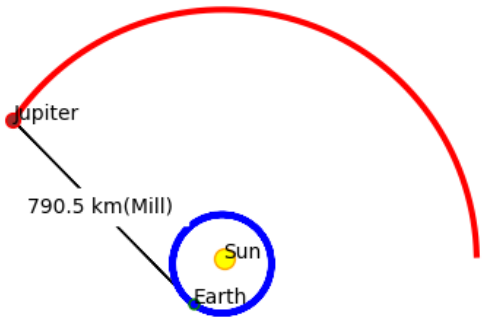


Figure 12. Example '1' - Orbits of Planets and distance between them.

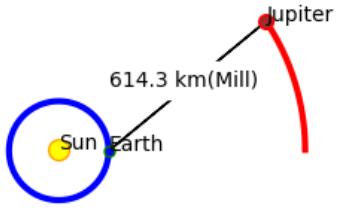


Figure 13. Example '2' - Orbits of the Planets and Distance between them.

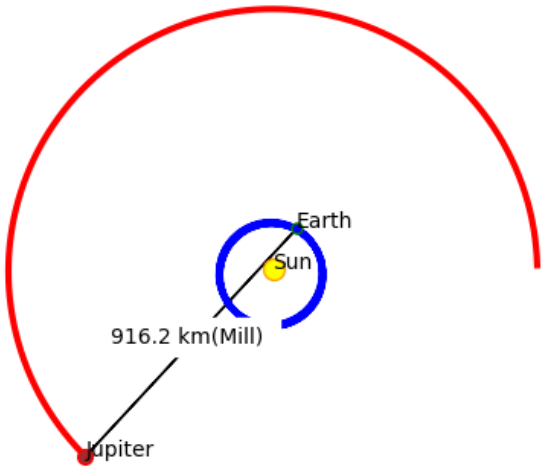


Figure 14. Example '3' - Orbits of Planets and Distance between them.

```
Total Orbital Observation Time (Days); 30
```

```
Total Orbital Observation Time (Days); 180
```

```
Total Orbital Observation Time (Days); 365
```

Figure 15. Total Number of Days the code is run. Example 1; 1 month, Example 2; 6 months, Example 3; 1 Year.

```
Total Orbital Observation Time (Days); 1825
```

Figure 16. Total Number of Days the code is run. Example 4; 5 Years.

```
Total Orbital Observation Time (Days); 9125
```

Figure 17. Total Number of Days the code is run. Example 5; 25 Years

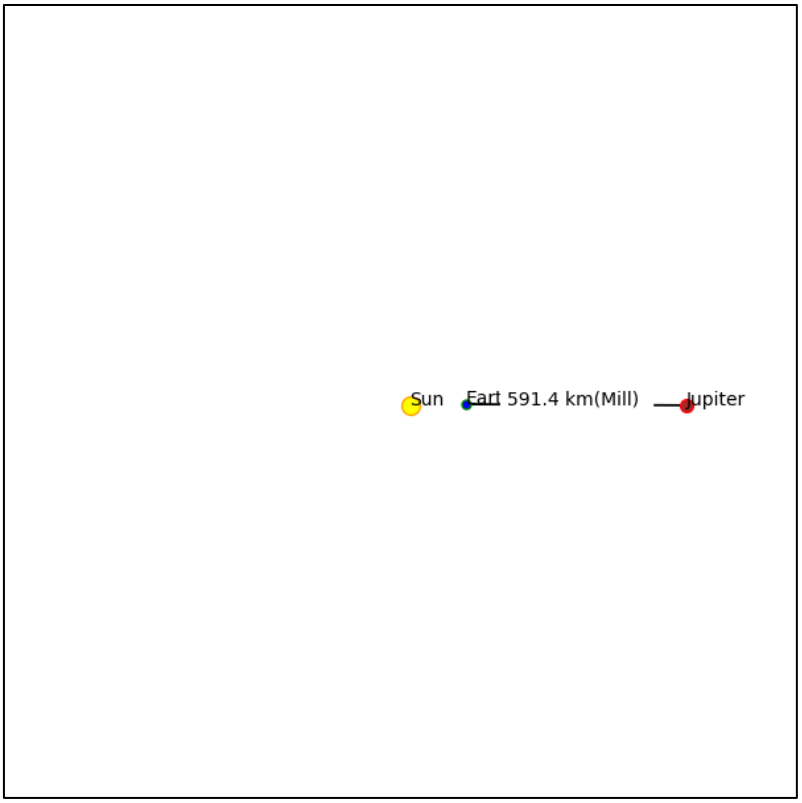
Years	Days since last Closest Approach (26/09/2022)	Approx. Distance between Planets (Mill*km)
1	403	595.8
2	803	611.8
3	1202	633.1
4	1599	652.4
5	1994	664.3
6	2390	665.6
7	2786	656.0
8	3184	638.0
9	3584	616.5
10	3986	598.7
11	4389	591.4
12	4791	597.5
13	5191	614.5
14	5589	635.7
15	5986	654.2
16	6381	664.8
17	6777	664.9
18	7173	654.4
19	7572	635.9
20	7972	614.6
21	8374	597.6
23	8777	591.6
24	9179	598.9

Figure 18. Screenshot of Plotted Output estimated date and distances of all Opposition Occurrences over next 25 Years.

	Top 5 Minimum Distances of Opposition (Approx. km)	Days Since Last Closest Approach
1	591.4	4389
2	591.6	8777
3	595.8	403
4	597.5	4791
5	597.6	8374

Figure 19. Screenshot of Top 5 Shortest Oppositions over next 25 Years.

Using this estimated date of closest approach produced by the code, the user need only to input the number of days printed into the interactive online website tool[18]. The start date to be input is of course 26/09/2022 and the 'Add' function is used along with an input of 4389 number of days as output by the code. See '*Figures '20'* and *'21'*, for screenshots of this process.




```
Estimation of Closest Aproach Date (Days Since Closest Approach); 4389
```

Figure 20. Outputs for Determination of Closest Approach over the next 25 years.

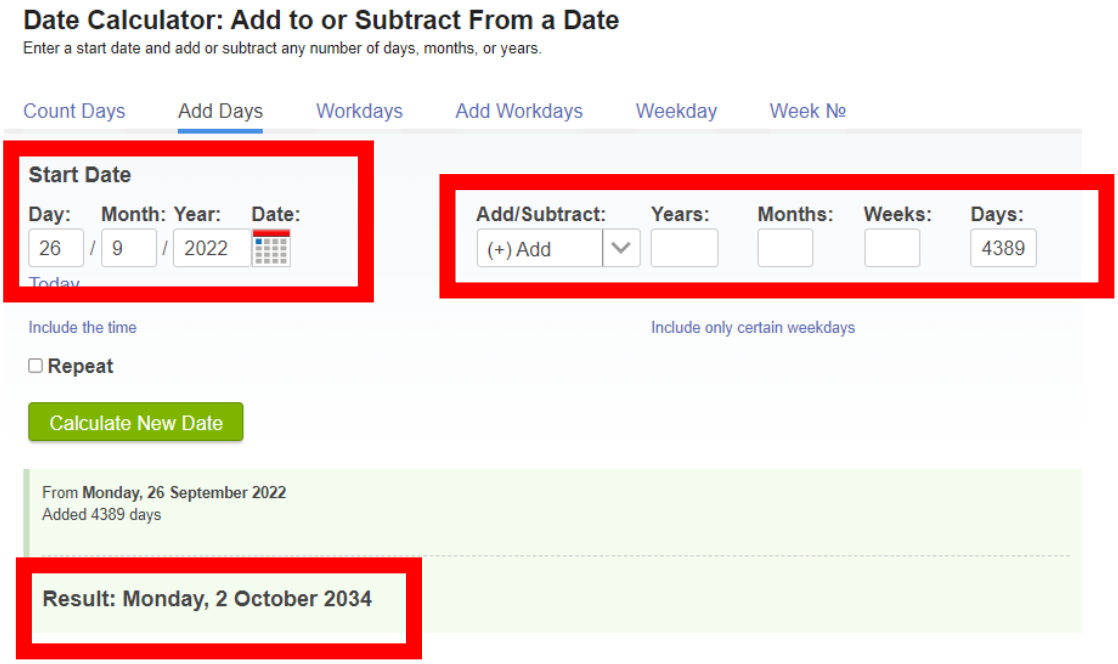


Figure 21. Screenshots of date determination from exact date of Closest Approach.

When this date is input into the '*Appendix 1 – Shell Script Used – 2. JPL Horizons Data'* code line '*sim_start_date ='* in the format*; "YYYY-MM-DD*". The duration of the simulation is irrelevant here, as the plot output for the simulation is not needed, only the one day of closest approach. This is required as it will provide verification of the 'Manual Data' code. If the planets were aligned in opposition at a minimum distance this proved the accuracy of the date determination of the original code. The plot output for an input date of '*sim_start_date = "2034-09-28"*' and a '*sim_duration*' of only 1, for 1 days output, is shown in '*Figure 22*' below.

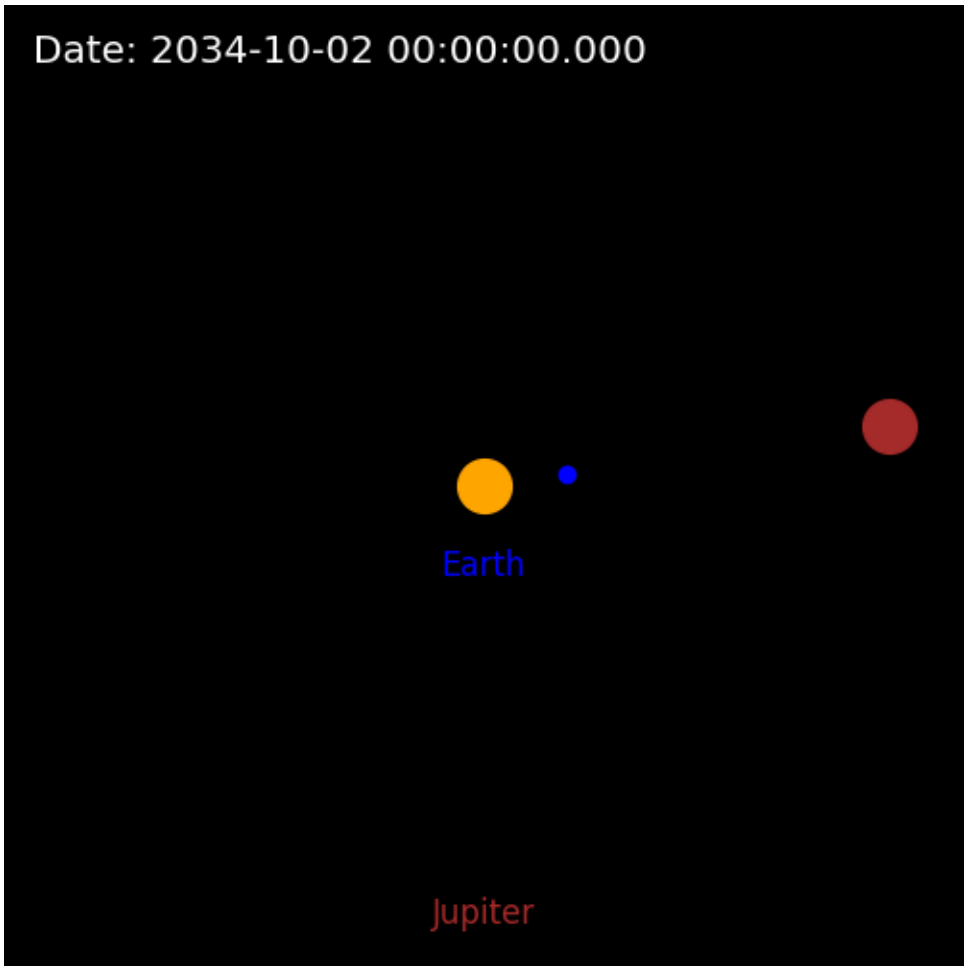


Figure 22.
*JPL Horizons Data interpretation of the Date of Closest Approach produced by the Manual Data code. (Clear Opposition and Minimum Distance between planets_*

Hohmann Transfer Trajectory at Closest Approach

Using the plot output from both '*Figures* '*20*' and '*22*', and '*equations*' '*1-9*', the phase angle of closest approach; also known as the 'critical angle', was determined to be 0.67834°. This estimation of critical angle or transfer orbit was confirmed by mathematics using Kepler's 1st Law. The results shown by running '*Appendix 1 – Shell Script Used – 3.*

Hohmann Transfer Trajectory' are shown by '*Figures* '*23*', '*24*', '*25*' and '*26*'. As can be seen by these figures, the path of the transfer takes a steady elliptical trajectory and meets a rendezvous at the exact moment of closest approach on the 2nd of October 2034.

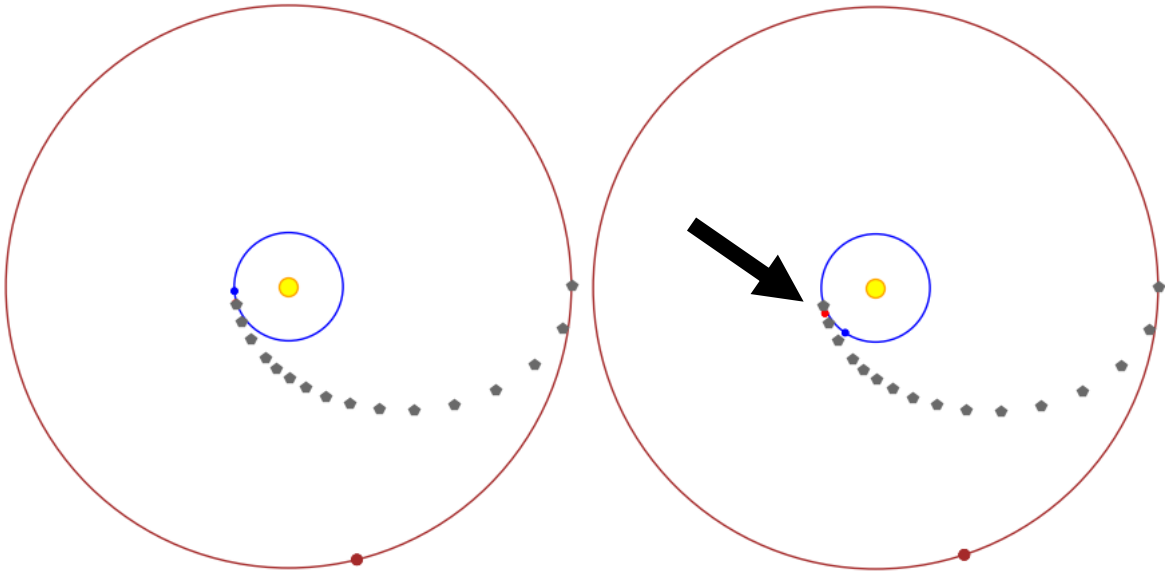


Figure 23. Start of Trajectory for Hohmann Transfer at Closest Approach.

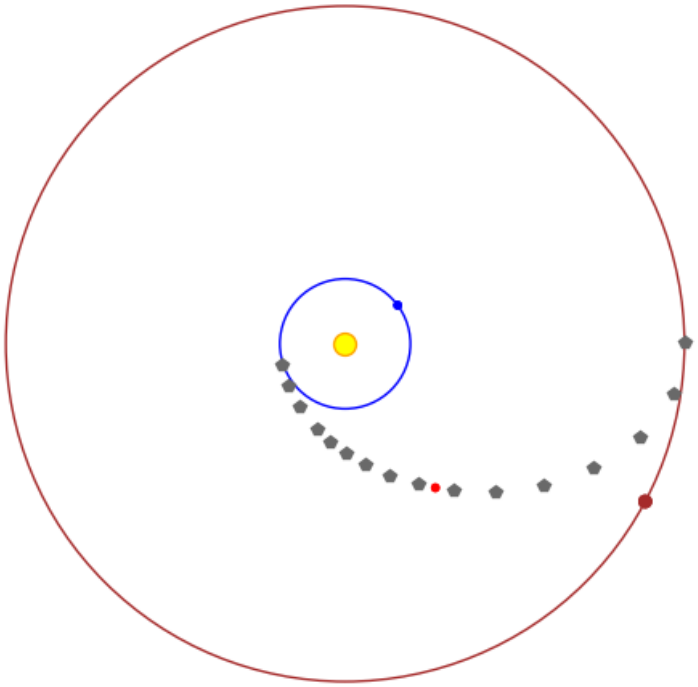


Figure 24. Mid-point of Trajectory for Hohmann Transfer at Closest Approach.

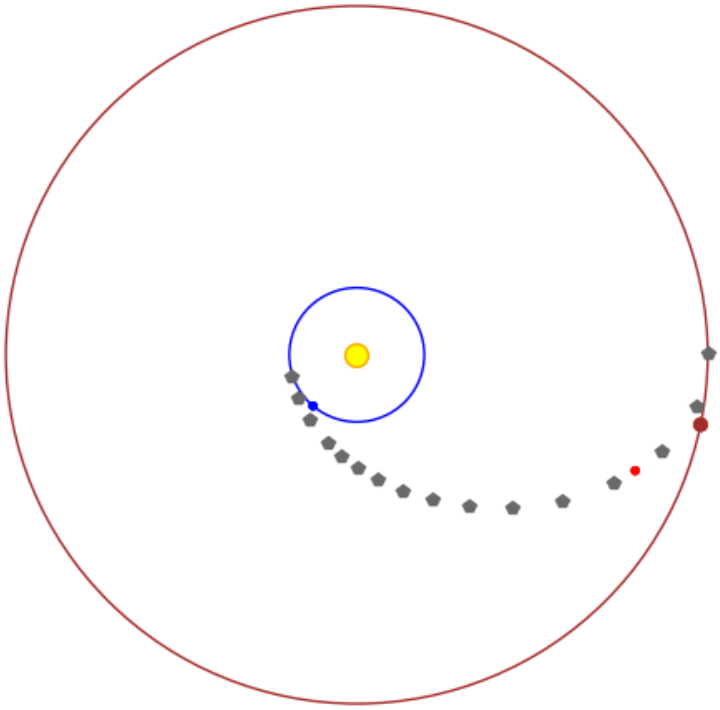


Figure 25. Late stage of Trajectory for Hohmann Transfer at Closest Approach.

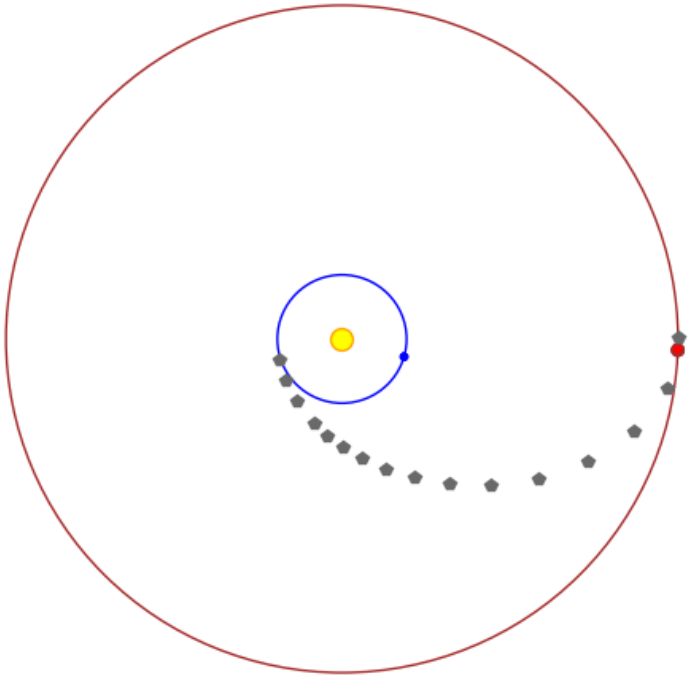


Figure 26. Completed Trajectory of Hohmann Transfer at Closest Approach.

These results show that the Hohmann Transfer uses the correct critical angle and launch date to arrive at rendezvous at the absolute correct moment, Therefore the Hohmann Transfer code executed the transfer to the perfect parameter suited to the Closest

Approach point as indicated in the first stage. See '*Table 2*', for complete summary of example Hohmann Transfer[11] parameters suited to transfer;

Orbital radius	5.204 AU
Orbital period	4332.6 d
Orbital velocity	13.06 km/s
Launch velocity	38.58 km/s
Launch burn	8.79 km/s
Transfer orbit (s-m axis)	3.102 AU
Transfer orbit eccentricity	0.678
Transfer time	997.8 d
Arrival velocity	7.41 km/s

Table 2. Example Hohmann Transfer Parameters.

The 'Ideal' Rocket.

The code used for the Ideal Rocket Equation provided the outputs shown in '*Figures '27' and '28'* below. These are useful for determining the rocket parameters required by the launch and transfer.

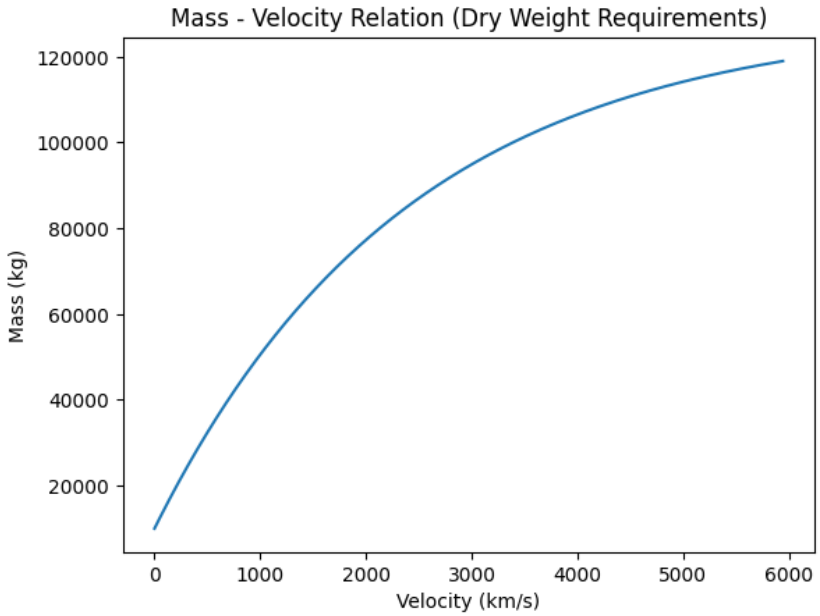


Figure 27. Plot Output – 'Mass - Velocity Relation' of Saturn V.

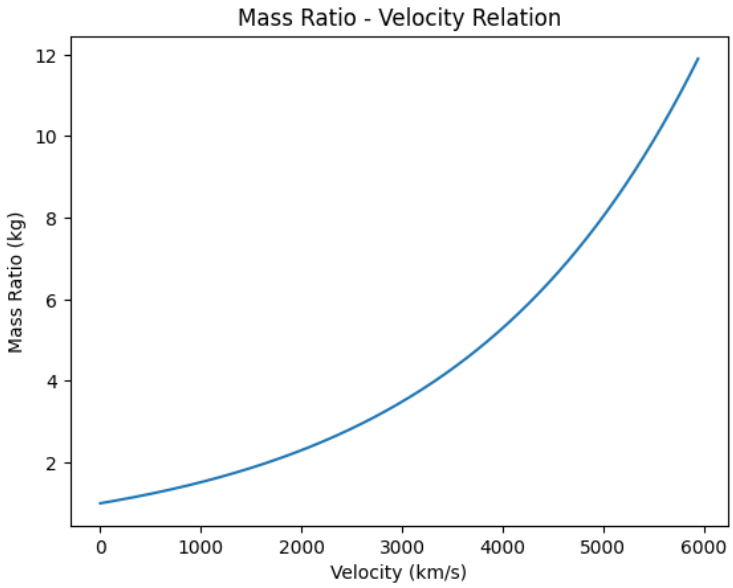


Figure 28. Plot Output – 'Mass Ratio – Velocity Relation' of Saturn V.

4.2 Determination of Optimal Transfer and Launch Window

Launch date would need to be approximately 997.8 days prior to the date of next closest approach for optimal transfer parameters. This would mean that the rocket must be launched 2.7343 years prior, on Thursday the 8th of January 2032. Launching on this date would provide the best window for fuel efficiency if Hohmann Transfer was used. However, if launching directly from the Earth to Jupiter in a straight-line direct trajectory; whereby fuel cost and payload size are considered inconsequential, the launch date would be altered. Seeing as NASA and ESA are increasingly dependent on SpaceX rockets as time progresses, let's assume for fuel efficiency and assumption into the future of spaceflight that the spacecraft that would be used for a mission such as this would be similar to SpaceX's current most powerful rocket, Starship, See '*Figure 29'*, which is expected to travel to Mars and potentially beyond!

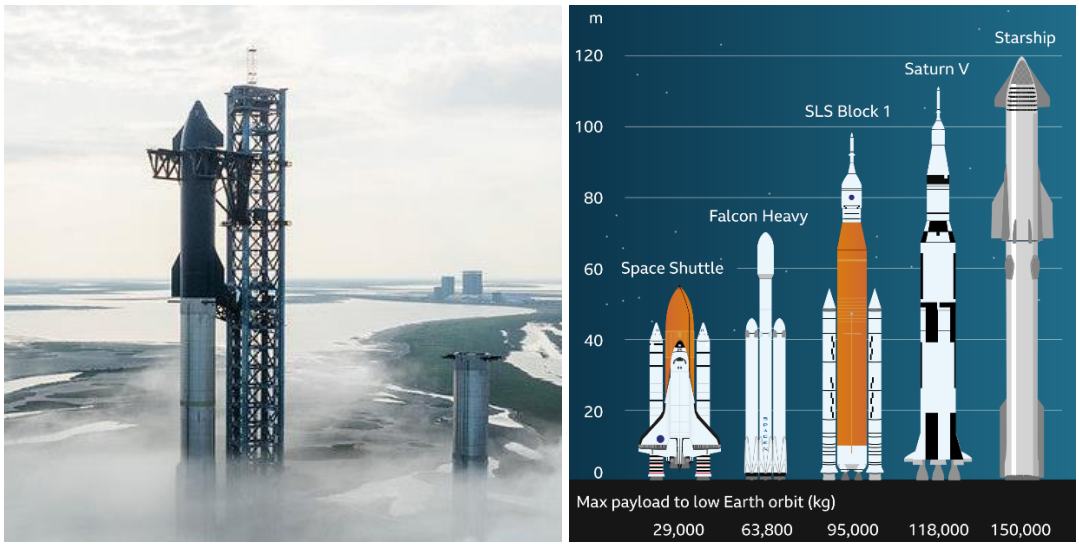


Figure 29. SpaceX's Starship Rocket & Payload Comparison to Other Spacecraft, including Saturn V.

It is expected that this rocket will travel at an average velocity of 39,600 kilometres per hour. To determine the time of flight assuming constant original burn, we need only consider the distance of closest approach; 591.4 million kilometres and divide in respect to velocity. This produces at flight time of approximately 14,934.34 hours of flight for a 'direct route'. This is equivalent to 622.26 days or 1.7 years. Therefore, for a 'direct flight' at closest approach using the SpaceX Starship rocket, the optimal launch window would need to be 622.26 days prior to the 2nd of October 2034, which would be on the 17th of January 2033. This is obviously a much quicker means of interplanetary space travel than using a Hohmann Transfer, however it requires far more rocket fuel and travel expenditure, even with an extremely powerful rocket. For example, if the rocket used was hydrogen based with a lower mass, such as Saturn V, moving at an average velocity of 40,234 kilometres per hour then it would likely have a shorter transfer time if sent on a direct trajectory at closest approach and therefore the rocket launch would have to account for 14699,01 hours of flight time using direct route. This is equivalent to 1.68 years which is slightly shorter than that of Starship.

Therefore, for convenience it would make more sense to rely on the older hydrogen-based rocket, though it may be smaller and far less powerful than the 'new and improved' version. Saturn V would make a suitable rocket type here, AS especially for a Hohmann Transfer as it doesn't require too much energy or rocket fuel, just enough to escape Earth's gravitational force and into a circular orbit about the Earth and then another burst of energy to allow it to ease into Jupiter's orbit. The remaining trajectory is completed using the transfer orbit.

4.3. Effects of a Delayed Launch

A delayed launch would greatly affect the Hohmann Transfer trajectory, this is due to the critical angle of the trajectory at launch date being altered and therefore increasing the error window. Obviously for the Hohmann Transfer investigated here, whereby the

spacecraft reaches the rendezvous point at the exact critical point of closest approach all the parameters are expected to be met; such as critical angle at launch date, launch at optimal launch window date, correct velocity adjustments – to bring spacecraft out of Earth orbit and also to return spacecraft into Jupiter's new orbit at its new radius. If all these parameters are met, then the transfer would be a success. However, if the launch date is affected this will either require more fuel consumption for course correction to place the rocket back on the original transfer trajectory. This would be extremely difficult and costly to the mission and would be best avoided or failing that; the rendezvous point would have to be reconsidered.

'*Figure 30*' shows an example of the same Hohmann Transfer considered previously, assuming the rendezvous point of closest approach remains the same for a delayed launch. The spacecraft would miss the rendezvous point without adjustment and Jupiter would pass by it in its orbit. As can be seen from the plot outputs of the trajectory of the transfer, the launch date is delayed by increasing the critical angle. In turn, the spacecraft only enters Jupiter's orbit after the planet has passed that point in its orbital path.

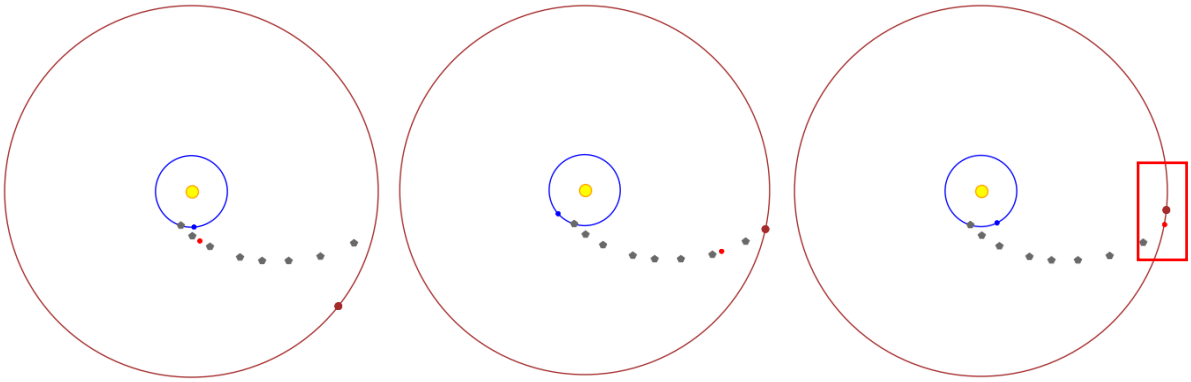


Figure 30. Plotted output of trajectory for Hohmann Transfer of Delayed Launch (Increased Critical Angle of 10° at an input of 0.678°)

By increasing the time increment of a delayed launch date, the trajectory of the transfer is increasingly affected, and therefore the gap of the idealised rendezvous at closest approach is further affected. The critical angle for launch is increased by increments of 10°. This is equivalent to increasing the delay by approximately 3.6 months per increment. '*Figure 31*' displays a zoomed in shot of the missed rendezvous point.

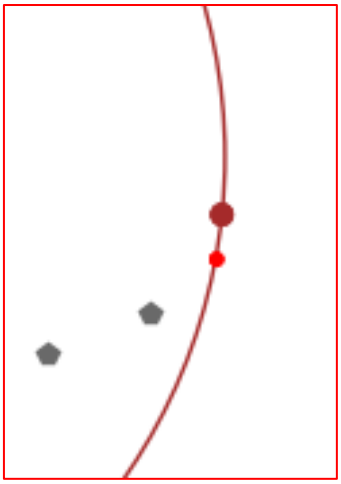


Figure 31. Visually Zoomed-in Shot of the Missed Rendezvous Point for Closest Approach.

4.4 Heavy Payload Delivery on a Hydrogen based Rocket. (>20 t)

Obviously, as the world moves in the direction of a much cleaner and environmentally friendly age of space exploration[25,26], the use of rocket fuel is growing increasingly unpopular. However, it is clear that the future of rocket travel cannot survive on solar power or electric battery-operated systems alone. The use of chemical propellant is necessary to generate the incredible boosts of thrust required by the rocket to escape the clutches of Earth gravity. Liquid hydrogen fuel is readily available and can be efficiently produced as a petroleum by product or even through electrolysis. Whereby electricity is used to split water into hydrogen and oxygen. Liquid hydrogen fuel has many benefits, mainly; its low molecular weight and high energy output when burned as a propellant with liquid oxygen. Liquid fuels tend to be the popular choice for secondary stages after solid fuels generate the required thrust for lift-off. Hydrogen fuel also allow low-density liquid fuel for the navigation thrusters when in orbit, this would be ideal for the Hohmann Transfer discussed. Whereby the thrusters would be used to change the spacecrafts velocity only twice once escaping the Earth's orbit, and then again to join Jupiter's orbit.

Hydrogen based rockets have been used for decades, dating back to NASA's early Apollo missions. It was the fuel of choice for the Saturn rockets for their secondary stage engines and also for the infamous Space Shuttle fleets rockets to power all three of their main rocket engines during its missions. The specific requirements of a space exploration mission are the main consideration when selecting the chemical propellant. For example, there may be different considerations taken for a manned mission to Jupiter versus an unmanned mission, this is of course, dependant on the orbital transfer manoeuvre of the mission.

The Saturn V rocket type had three different stages, the first stage lifted the rocket to an altitude of approximately 68 kms, the second carried the rocket from that altitude to the higher orbit, and then in the final stage the rocket is sent into an Earth orbit, meanwhile

the remaining first and second stage rocket parts fell back towards Earth eventually burning and landing in the ocean. The remaining command module and 3rd Stage rocket engine could potentially carry enough fuel for the critical bursts of energy required to get out of the Earth's circular orbit, into the elliptical transfer orbit and then the final burst required to join Jupiter's circular orbit. With some small modifications, this rocket type could potentially travel the trajectory discussed here.

As Jupiter is a gaseous planet and has a gas composition of approximately 75% hydrogen, this planet may be perfect if the mission required a return trip, obviously the launch date isn't for at least 10 years, and that could be sufficient time to create some sort of mechanism that could extract the atmospheric hydrogen surrounding Jupiter's moons and possibly convert it into liquid hydrogen which could refuel the third stage rocket engine or similar to secure a return trip of the Saturn V rocket command module. Of course, there is the problem of Jupiter's massive gravitational force on the rocket which could refrain it from doing this, but that's a topic for another project.

At launch this rocket has a mass of approximately 2812.27 tonnes. After stages 1 and 2, the gross mass of the remaining rocket engine and was estimated at approximately 123 tonnes. This is a much higher mass than the required >20t of the investigation, therefore it is completely feasible to send a rocket such as Saturn V or Starship, assuming they follow the optimal Hohmann Trajectory and stages outlined and utilise their propellant correctly with optimal efficiency.

5. Discussion of Conclusions.

5.1 Suitability of Optimal Transfer Window for Jupiter

Based on the results found in '*4.2 -Determination of Optimal Transfer and Launch Window'*, it may be concluded that 8th of January 2032; is the perfect transfer window launch date, by means of a Hohmann Transfer at which the Saturn V command module and stage 3 engine which had a gross mass of approximately 123 tonnes or a heavy weight hydrogen-based mini rocket, could travel to Jupiter. This is mainly due to the launch date providing an optimal window at which the rocket could complete the transfer without use of excess fuel burn and course correction. This window would provide the most fuel efficient and minimum distance required of the transfer and would therefore be deemed suitable.

However, if the launch date was delayed, course correction would most definitely be required, which would come at a high and complicated cost of fuel burn and trajectory. If not completed, the rendezvous of the mission would be difficult to achieve at closest approach and the spacecraft may miss Jupiter in its orbit as seen by '*Figure 31. Visually Zoomed-in Shot of the Missed Rendezvous Point for Closest Approach'.*

Of course, as aforementioned, the 'Optimal Transfer Window' here, is of course the next date of closest approach within the next 25 years, the 2nd of October 2034. If this optimal transfer window is met, it would be the most fuel efficient and suitable window for years to come.

5.2 Practicality of Heavy Payload Delivery

From the results displayed by '*4.4 Heavy Payload Delivery on a Hydrogen based Rocket. (>20 t)'*, it may be deemed completely feasible and possible in terms of practicality to launch a heavy payload delivery, such as the Saturn V command module and Stage 3 rocket engine, weighing at approximately 123 tonnes, along the Hohmann Trajectory created, to rendezvous at moment of closest approach.

In fact, seeing as Jupiter consists of 75% atmospheric hydrogen, it may be possible to plan a return trip of the rocket if it was somehow possible to extract and convert the moon's atmospheric hydrogen into liquid propellant for the rocket. Therefore, it is definitely practical to send a heavy payload delivery which could be refilled and returned to Earth if needed.

5.3 Conclusions & Summary of Results

In conclusion, this project investigation may be deemed an overall success as the overall aims and objectives were met. The modelling of relative positions of Earth and Jupiter in their orbit to the sun over a specific period of 25 years (2022-2047) was completed using Python visualisation and animation. This data was used to estimate the optimal transfer windows and orbits, inclusive of time and velocity changes required, for a successful Hohmann Transfer. The trajectory and results of the optimal transfer window allowed investigation into the effects on the mission if launch is delayed from ideal launch window. This found that course correction would be required at a fuel cost, as failing to do so would mean the trajectory of the Hohmann transfer would miss the ideal rendezvous at closest approach. From these results, it was simple to decide on a logical determination on whether it is realistic to deliver a payload of more than 20t into an orbital transfer with a liquid hydrogen-based rocket. Results found that not only is it logical and feasible, but it would also be a perfect opportunity to allow the rocket to make a return trip by possibly refuelling using and converting Jupiter's atmospheric hydrogen to liquid propellant for the rocket. A suitable rocket example would be to use Saturn V, whereby the command module and stage 3 engine may be used be to complete this transfer which has a payload size of approximately 123 tonnes.

References

Text & Informational References;

Websites;

1. Unauthored, Mars Exploration Program and the Jet Propulsion Laboratory for NASA's Science Mission Directorate, NASA, 8th Dec 2022, 'Trip to Mars', Website; https://mars.nasa.gov/mars2020/timeline/cruise/Moons

2. Unauthored, Planetary Science Communications team at NASA's Jet Propulsion Laboratory for NASA's Science Mission Directorate, Phil Davis & Steve Carney, NASA, 10th April 2023, 'Jupiter Moons', Website; https://solarsystem.nasa.gov/planets/jupiter/in-depth/

3. Miller K, Scientific American, 28th Oct 2021, 'Jupiter's Great Red Spot Is Surprisingly Deep', Website; https://www.scientificamerican.com/article/jupiters-great-red-spot-is-surprisingly-deep/

4. Mundell C, ESA, 30th March 2023, 'JUICE LAUNCH KIT', Website; https://esamultimedia.esa.int/docs/science/Juice-LaunchKit.pdf

5. Unauthored, ESA, Undated, 'Jupiter Icy Moons Explorer', Website; https://www.esa.int/Science_Exploration/Space_Science/Juice

6. Greicius T, NASA, 13th Feb 2015, 'Juno - Mission to Jupiter', Website; https://www.nasa.gov/mission_pages/juno/main/index.html

7. Nelson J, Caltech-JPL, NASA, Undated, 'Voyager spacecraft', Website; https://voyager.jpl.nasa.gov/mission/

8. Ridgeway B, NASA, 16th Sept 2022, 'Jupiter to Reach Opposition, Closest Approach to Earth in 59 Years!', Website; https://blogs.nasa.gov/Watch_the_Skies/2022/09/16/jupiter-to-reach-opposition-closet-approach-to-earth-in-70-years

9. Schreiber B A, Encyclopædia Britannica, 29th March 2023, 'Kepler's Laws of Planetary Motion', Website; https://www.britannica.com/science/Keplers-laws-of-planetary-motion

10. Zhu A, Medium, 10th June 2022, 'Simulate a Tiny Solar System with Python', Website; https://towardsdatascience.com/simulate-a-tiny-solar-system-with-python-fbbb68d8207b

11. Giesen J, Applet, 7th Oct 2015, 'Applet: Hohmann Transfer Orbit', Website; http://www.jgiesen.de/hohmann/

12. Carney S & Davis P, Planetary Science Communications team at NASA's Jet Propulsion Laboratory for NASA's Science Mission, Directorate, 'Basics of Space Flight - Solar System Exploration: NASA Science - Hohmann Transfer', Undated, Website; https://solarsystem.nasa.gov/basics/chapter4-1/

13. Heita P, Medium, 28th Dec 2021, 'Simulate Interplanetary Space Travel in Python', Website; https://medium.com/python-in-plain-english/simulating-interplanetary-space-travel-in-python-95116b14b6a9

14. Boucher D, Exoplanet Science, 6th Jan 2023, 'How Long Would A Hohmann Transfer To Jupiter Take', Website; https://exoplanetscience.org/how-long-would-a-hohmann-transfer-to-jupiter-take

15. Unauthored, The Physics Classroom, March 2004, 'Newton's Law of universal gravitation', Website; https://www.physicsclassroom.com/class/circles/Lesson-3/Newton-s-Law-of-Universal-Gravitation

16. Hall N, NASA, 27th Oct 2022, 'Newton's Second Law of Motion', Website; https://www1.grc.nasa.gov/beginners-guide-to-aeronautics/newtons-laws-of-motion/

17. Park R, NASA's Jet Propulsion Laboratory, Undated, 'Horizons System', Website; https://ssd.jpl.nasa.gov/horizons/

18. Thorsen S, Time and Date AS 1995-2023, 'Days Calculator: Days Between Two Dates', Website; https://www.timeanddate.com/date/duration.html

19. Ashika L, Save My Exams, March 2019, 'Orbital periods (8.1.4): Edexcel IGCSE Physics Revision Notes 2019', Website; https://www.savemyexams.co.uk/igcse/physics/edexcel/19/revision-notes/8-astrophysics/8-1-motion-in-the-universe/8-1-4-orbital-period/

20. Liu J, Medium, 13th Oct 2020, 'It's Not Like It's Rocket Science', Website; https://medium.com/predict/its-not-like-it-s-rocket-science-b89d0cb8c767

21. Benson T, NASA, 13th May 2021, 'Ideal Rocket Equation', Website; https://www.grc.nasa.gov/www/k-12/rocket/rktpow.html

22. May S, NASA, 7th Aug 2017, 'What Was the Saturn V?', Website; https://www.nasa.gov/audience/forstudents/5-8/features/nasa-knows/what-was-the-saturn-v-58.html

23. Unauthored, SpaceX, Undated, 'Starship', Website; https://www.spacex.com/vehicles/starship/

24. Carney S, Planetary Science Communications team at NASA's Jet Propulsion Laboratory for NASA's Science Mission Directorate, NASA, 17th Oct 2022, 'Jupiter – In Depth', Website; https://solarsystem.nasa.gov/planets/jupiter/in-depth/

25. Zona K & Dunbar B, NASA, 29th July 2010, 'Liquid Hydrogen--the Fuel of Choice for Space Exploration', Website; https://www.nasa.gov/topics/technology/hydrogen/hydrogen_fuel_of_choice.html

26. Whitaker M, Fortune, 6th Dec 2022, 'Meet the companies revolutionizing cleaner rocket fuel alternatives', Website; https://fortune.com/2022/12/05/space-travel-is-heating-up-and-so-are-rocket-fuel-emissions-these-companies-are-developing-cleaner-alternatives-to-protect-earth-first/

Books;

- 'Basics of Spaceflight; for space exploration, space commercialization, and space colonization, Dr. Regina E. Schulte-Ladbeck, RESLscience, Publication Year; 2016.

- 'University physics', Young & Freedman, Pearson, Publication Year; 2019.

Figure References.

Figure 1. NASA, ESA, CSA, Jupiter ERS Team; image processing by Ricardo Hueso (UPV/EHU), Alise Fisher and Judy Schmidt, JWT, 22nd Aug 2022, 'Jupiter System' https://blogs.nasa.gov/webb/2022/08/22/webbs-jupiter-images-showcase-auroras-hazes/

Figure 2. NASA, 7th Aug 2017, 'Jupiter's Great Red Spot' https://www.nasa.gov/multimedia/imagegallery/image_feature_413.html

Figure 3. NASA, Undated, 'Juno', https://www.missionjuno.swri.edu/spacecraft/juno-spacecraft

Figure 4. NASA, Undated, 'Voyager', https://voyager.jpl.nasa.gov/assets/images/galleries/Spacecraft_Profile.jpg

Figure 29. Two images;

Official SpaceX Photos, 9th Jan 2023, Flicker, 'Starship Full Stack',
https://www.flickr.com/photos/spacex/52621815136/

Jonathan Amos, 9th Feb 2023, BBC News,
https://www.bbc.com/news/science-environment-64590147

Appendices

Although, I am overall extremely pleased and proud of this report and the investigation made within it, there are a few small things I would've loved to do if time allowed. For example, I would have included a comparison of data between the '*Manual Orbit*' code output and the '*JPL Horizons'* code. I also would have loved to explore the possibility of sending a manned mission along the Hohmann Transfer discussed and dictate the changes this may have on the payload size and fuel requirements, mainly due to the very real possibility of a crew potentially making this mission within my lifetime. Lastly, I wish I could've had more time to really delve into the orbital mechanics and the different transfer methods and rocket parameters required to complete a mission to Jupiter if cost was of no issue. Of course I also wish I had have managed my time better and paid more attention to detail during report completion. That said, I am satisfied that I have gone to extreme lengths to meet the aims of this project and can honestly admit that I am proud of my work.

Appendix 1. Shell Script Used

The full script may be found, used and run on the Google Colab Notebook, found at the following; Colab Notebook website.

Appendix 1 – Shell Script Used – 1. 'Manual Data - Closest Approach'.

```
# -*- coding: utf-8 -*-
"""Manual Orbit.ipynb

Automatically generated by Colaboratory.

Original file is located at
    https://colab.research.google.com/drive/1kUSSu4ujIzj9fQ2Vl-KL2KMSVTde7DdR
"""

pip install --upgrade matplotlib==3.5.3

import warnings
import matplotlib.cbook
warnings.filterwarnings("ignore",category=matplotlib.cbook.mplDeprecation)

import matplotlib.pyplot as plt
import matplotlib as mpl
from matplotlib import animation
```

```
import matplotlib.table
matplotlib.rcParams['animation.embed_limit'] = 2**1023
from IPython.display import HTML
from math import sqrt
mpl.rcParams.update(mpl.rcParamsDefault)
#\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\

# First to define the Constants
G          = 6.67e-11              # Gravitational Constant
Ms         = 1.988e30              # Mass of Sun (kg)
Me         = 5.972e24              # Mass of Earth (kg)
Mj         = 1.898e27              # Mass of Jupiter (kg)
AU         = 1.49598e11            # Astronomical Unit in (m)
daysec     = 24.0*60*60            # hours*mins*secs
dt         = 1*daysec              # Time difference
t          = 0.0                   # Initial time

e_ap_v     = 29780                 # Earth's velocity
j_ap_v     = 13060                 # Jupiter's velocity

gc_e = G*Me*Ms                     # Gravitational Const on Earth
gc_j = G*Mj*Ms                     # Gravitational Const on Jupiter

# To setup the starting conditions and locations on plot of each planet;

# Earth
xe,ye    = (1*AU),0                # x,y-position on plane (Average AU distance of Earth from Sun)
yve,xve  = e_ap_v,0                # x,y-positional velocity
# Jupiter
xj,yj    = (5.2*AU),0              # x,y-position on plane
yvj,xvj = j_ap_v, 0                # x,y-positional velocity

# Sun
xs,ys   = 0,0                      # Centroid Position of Sun
xvs,yvs = 0,0                      # Stationary zero velocity of Sun

xelist,yelist = [],[]              # Orbital Positional Data of Earth
xjlist,yjlist = [],[]              # Orbital Positional Data of Jupiter
xslist,yslist = [],[]              # Stationary Position Data of Sun

# start simulation
while t<1*30*daysec:               # Input value of time observation
   ###### Earth ######

   # Force Vector // G force on Earth / F = -G[Mmr/r**3]r
   rx,ry = xe - xs, ye - ys        # Radius positional vector; r
   cube_of_r_e   = (rx**2+ry**2)**1.5   # r vector cubed
   fx_e = -gc_e*rx/cube_of_r_e     # Gravitational force on Earth in x-direction
```

```
    fy_e = -gc_e*ry/cube_of_r_e        # Gravitational force on Earth in y-direction

    # Velocity Vector on Earth / F = ma -> a = F/m
    xve += fx_e*dt/Me                  # Velocity Vector in x-direction after time difference
    yve += fy_e*dt/Me                  # Velocity Vector in y-direction after time difference

    # update position for displacement of Earth // velocity x time difference = distance
    xe += xve*dt                       # Displacement x-position
    ye += yve*dt                       # Displacement y-position

    # save the position in list
    xelist.append(xe)                  # Adding new y-positional data to list
    yelist.append(ye)                  # Adding new y-positional data to list

    ######## Jupiter ########

     # Force Vector // G force on Jupiter / F = -G[Mmr/r**3]r
    rx_j,ry_j = xj - xs, yj - ys       # Radius positional vector; r
    cube_of_r_j = (rx_j**2+ry_j**2)**1.5 # Cube of r Vector
    fx_j = -gc_j*rx_j/cube_of_r_j      # Gravitational Force on Jupiter in x-direction
    fy_j = -gc_j*ry_j/cube_of_r_j      # Gravitational Force on Jupiter in y-direction
    xvj += fx_j*dt/Mj                  # Velocity Vector in x-direction after time difference
    yvj += fy_j*dt/Mj                  # Velocity Vector in y-direction after time difference

    # update position for displacement of Jupiter // velocity x time difference = distance
    xj += xvj*dt                       # Displacement x-position
    yj += yvj*dt                       # Displacement y-position

    # add new positional data to list
    xjlist.append(xj)                  # Adding new x-positional data to list
    yjlist.append(yj)                  # Adding new y-positional data to list

    ###### Sun ######
    # update quantities how is this calculated?  F = ma -> a = F/m
    xvs += -(fx_e+fx_j)*dt/Ms          # Zero ... Velocity Vector in x-direction after time difference
    yvs += -(fy_e+fy_j)*dt/Ms          # Zero ... Velocity Vector in x-direction after time difference

    # update position
    xs += xvs*dt                       # Zero ... Displacement x-position
    ys += yvs*dt                       # Zero ... Displacement y-position
    xslist.append(xs)                  # Zero ... Adding new x-positional data to list
    yslist.append(ys)                  # Zero ... Adding new x-positional data to list

    # update dt
    t +=dt                         # Increment of Time Difference

# To animate Data for Observation
fig, ax = plt.subplots(figsize=(6,6))
```

```
ax.set_aspect('auto')
ax.set_facecolor("w")
plt.axis('off')

# Defining the planets and orbit path attributes in title, shape, size & colour.

# Earth on plotted animation
line_e,     = ax.plot([],[],'b',lw=3,)
point_e,            = ax.plot([AU], [0], marker="o", markersize=5, markeredgecolor="green", markerfacecolor="blue")
text_e      = ax.text(AU,0,'Earth')
# Jupiter on plotted animation
line_j,     = ax.plot([],[],'r',lw=3)
point_j,            = ax.plot([5.2*AU], [0], marker="o", markersize=7, markeredgecolor="red", markerfacecolor="brown")
text_j      = ax.text(5.2*AU,0,'Jupiter')
# Sun on plotted animation
point_s,            = ax.plot([0], [0], marker="o", markersize=10, markeredgecolor="orange", markerfacecolor="yellow")
text_s      = ax.text(0,0,'Sun')

# Connecting orbits to display distance vector between them
annotation  = ax.arrow(0, AU, 0, (5.2 + 1) * AU)         # Use of arrow annotion for line connection between planets
text_a      = ax.text(1, (5.2+ 1 ) / 2 * AU, '', ha='center', backgroundcolor='white') # Text box to display distace vector between planets

exdata,eydata = [],[]               # Earth track
sxdata,sydata = [],[]               # Sun track
jxdata,jydata = [],[]               # Jupiter Track

print("Total Orbital Observation Time (Days);",len(xelist))

# Creating an update loop of the planetary positional attributes and data
def update(i):

    exdata.append(xelist[i])
    eydata.append(yelist[i])

    jxdata.append(xjlist[i])
    jydata.append(yjlist[i])

    line_e.set_data(exdata,eydata)
    point_e.set_data(xelist[i],yelist[i])
    text_e.set_position((xelist[i],yelist[i]))

    line_j.set_data(jxdata,jydata)
    point_j.set_data(xjlist[i],yjlist[i])
```

```
    text_j.set_position((xjlist[i],yjlist[i]))

    point_s.set_data(xslist[i],yslist[i])
    text_s.set_position((xslist[i],yslist[i]))

    # Axis Limit of Plotted Orbits being Displayed
    ax.set_xlim(-5.5*AU,5.5*AU)
    ax.set_ylim(-5.5*AU,5.5*AU)

    dist = sqrt((exdata[-1] - jxdata[-1])**2 + (eydata[-1] - jydata[-1])**2)
    annotation.set_data(x=exdata[-1], y=eydata[-1], dx=jxdata[-1] - exdata[-1], dy=jydata[-1] -
eydata[-1])
    text_a.set_position(((exdata[-1] + jxdata[-1]) / 2, (eydata[-1] + jydata[-1]) / 2))
    text_a.set_text(f"{dist/1e9:.1f} km(Mill) ")

#data = [i,(f"{dist}")]
#print(min(data((f"{dist}"))))

    return line_e,point_s,point_e,line_j,point_j,text_e,text_j,text_s,
anim = animation.FuncAnimation(fig,func=update,frames=len(xelist),interval=1,blit=True)


from IPython.display import HTML
HTML(anim.to_jshtml())
```

Appendix 1 – Shell Script Used – 2. 'JPL Horizons Data'.

```
"""JPL.ipynb

Automatically generated by Colaboratory.

Original file is located at
    https://colab.research.google.com/drive/1RiaH7jiHlw-nyxilbb6hdfRWQhSxsr7W
"""

!pip install astroquery
!pip install astropy

from IPython.utils.contexts import NoOpContext
import decimal as dc
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.animation as animation
from decimal import Decimal
from astropy.time import Time
from astroquery.jplhorizons import Horizons
from matplotlib import rc
```

```
rc('animation', html='jshtml')


sim_start_date = "YYYY-MM-DD"    # simulating a solar system starting from this date
sim_duration = 365 # (int) # to run simulation for 25 years after starting point 01/01/2025
m_earth = 5.972e24 / 2e30  # Mass of Earth relative to mass of the sun

class Object:                # define the objects: the Sun, Earth, Jupiter, etc
   def __init__(self, name, rad, color, r, v):
      self.name = name
      self.r    = np.array(r, dtype=np.float64)
      self.v    = np.array(v, dtype=np.float64)
      self.xs = []
      self.ys = []
      self.plot = ax.scatter(r[0], r[1], color=color, s=rad**2, edgecolors=None, zorder=10)
      self.line, = ax.plot([], [], color=color, linewidth=1.4)

class SolarSystem:
   def __init__(self, thesun):
      self.thesun = thesun
      self.planets = []
      self.time = None
      self.timestamp = ax.text(.03, .94,'Date: ', color='w', transform=ax.transAxes, fontsize='x-large')
   def add_planet(self, planet):
      self.planets.append(planet)
   def evolve(self):         # evolve the trajectories
      dt = 1.0
      self.time += dt
      plots = []
      lines = []
      for p in self.planets:
         p.r += p.v * dt
         acc = -2.959e-4 * p.r / np.sum(p.r**2)**(3./2)  # in units of AU/day^2
         p.v += acc * dt
         p.xs.append(p.r[0])
         p.ys.append(p.r[1])
         p.plot.set_offsets(p.r[:2])
         p.line.set_xdata(p.xs)
         p.line.set_ydata(p.ys)
         plots.append(p.plot)
         lines.append(p.line)
      self.timestamp.set_text('Date: ' + Time(self.time, format='jd', out_subfmt='float').iso)
      return plots + lines + [self.timestamp]

plt.style.use('dark_background')
fig = plt.figure(figsize=[5, 5])
ax = plt.axes([0.,0.,1,1], xlim=(-5.8,5.8), ylim=(-5.8, 5.8))
```

```
ax.set_aspect('equal')
ax.axis(False)
ax.grid()
ss = SolarSystem(Object("Sun", 20, 'orange', [0, 0, 0], [0, 0, 0]))
ss.time = Time(sim_start_date).jd
colors = ['blue', 'brown']
sizes = [0.3, 1]
names = ['Earth', 'Jupiter']
texty = [1, 5.2]
for i, nasaid in enumerate([3, 5]):  # The 3rd and 5th planet in solar system order
   obj = Horizons(id=nasaid, location="@sun", epochs=ss.time, id_type='majorbody').vectors()
   ss.add_planet(Object(nasaid, 20 * sizes[i], colors[i],
                        [np.double(obj[xi]) for xi in ['x', 'y', 'z']],
                        [np.double(obj[vxi]) for vxi in ['vx', 'vy', 'vz']]))
   ax.text(0, - (texty[i] + 0.1), names[i], color=colors[i], zorder=1000, ha='center', fontsize='large')
def animate(i):

   return ss.evolve()

   fig.show()

ani = animation.FuncAnimation(fig, animate, repeat=False, frames=sim_duration, blit=True, interval=20,)
ani
```

Appendix 1 – Shell Script Used – 3. 'Hohmann Transfer Trajectory'

```
"""Hohmann Trasfer.ipynb

Automatically generated by Colaboratory.

Original file is located at
   https://colab.research.google.com/drive/1FoNcDdZjlmPaEXiaEdT9sCV6yCTkfAND
"""

pip install latex

!sudo apt-get install texlive-latex-extra texlive-fonts-recommended dvipng cm-super

# First to define the Constants
G         = 6.67e-11
Ms        = 1.988e30              # Sun
Me        = 5.972e24              # Earth
Mj        = 1.898e27              # Jupiter
AU        = 1.5e11
daysec    = 24.0*60*60

e_ap_v    = 29780              # Earth's velocity
```

```
j_ap_v     = 13060              # Jupiter's velocity


# At closest approach; Travel time to Jupiter;  997.8 days.  (32.77 Months) or (2.729056 Years).
# Rocket would leave Earth 998 days before outputted closest approach day.
# Launch Date for next closest approach
# Angle between planets position at launch date =

import numpy as np
import matplotlib as mpl
import matplotlib.pyplot as plt
from matplotlib import animation

# Plotting the Earth and Jupiter orbits
alpha = 186 #degrees (Angle by which should be ahead by)
Earth = plt.Circle((0,0), radius= 1.0,fill=False,color='blue')
Jupiter = plt.Circle((0,0), radius= 5.2,fill=False,color='brown')

# Moving Earth, Jupiter, and Spacecraft
patch_E = plt.Circle((0.0, 0.0),radius=0.05,fill=True,color='blue')
patch_J = plt.Circle((0.0, 0.0),radius=0.09,fill=True,color='brown')
patch_H = plt.Circle((0.0, 0.0),radius=0.05,fill=True,color='red')

def init():
 patch_E.center = (0.0,0.0)
 ax.add_patch(patch_E)
 patch_J.center = (0.0,0.0)
 ax.add_patch(patch_J)
 patch_H.center = (0.0,0.0)
 ax.add_patch(patch_H)
 return patch_E,patch_J,patch_H

def animate(i):
 #Earth
 x_E, y_E = patch_E.center
 x_E = np.cos((2*np.pi/365.2)*4*i)
 y_E = np.sin((2*np.pi/365.2)*4*i)
 patch_E.center = (-y_E, x_E)
#Jupiter
 x_J, y_J = patch_J.center
 x_J = 5.2*np.cos((2*np.pi/4332.6)*4*i+(np.pi*alpha/180.))
 y_J = 5.2*np.sin((2*np.pi/4332.6)*4*i+(np.pi*alpha/180.))
 patch_J.center = (-y_J,x_J)
#Hohmann
 Period = 1995.6
 x_H = 3.1*(1. - 0.678**2)/(1. + 0.678*np.cos((2*np.pi/Period)*4*i))*np.cos((2*np.pi/Period)*4*i)
 y_H = 3.1*(1. - 0.678**2)/(1. + 0.678*np.cos((2*np.pi/Period)*4*i))*np.sin((2*np.pi/Period)*4*i)
 patch_H.center = (-x_H,-y_H)
```

```
 return patch_E,patch_J,patch_H

plt.rcParams['savefig.bbox'] = 'tight'
Writer = animation.writers['ffmpeg']
writer = Writer(fps=200, metadata=dict(artist='Me'), bitrate=6000)

plt.rc('font', family='serif', serif='Times')
plt.rc('text', usetex=True)
plt.rc('xtick', labelsize=6)
plt.rc('ytick', labelsize=6)
plt.rc('axes', labelsize=6)

# Set up path, to guide eye
Period = 1995.6

x_H_A = 3.1*(1. - 0.678**2)/(1. + 0.678*np.cos((2*np.pi/Period*100)))*np.cos((2*np.pi/Period*100))
y_H_A = 3.1*(1. - 0.678**2)/(1. + 0.678*np.cos((2*np.pi/Period*100)))*np.sin((2*np.pi/Period*100))
x_H_B = 3.1*(1. - 0.678**2)/(1. + 0.678*np.cos((2*np.pi/Period*200)))*np.cos((2*np.pi/Period*200))
y_H_B = 3.1*(1. - 0.678**2)/(1. + 0.678*np.cos((2*np.pi/Period*200)))*np.sin((2*np.pi/Period*200))
x_H_C = 3.1*(1. - 0.678**2)/(1. + 0.678*np.cos((2*np.pi/Period*300)))*np.cos((2*np.pi/Period*300))
y_H_C = 3.1*(1. - 0.678**2)/(1. + 0.678*np.cos((2*np.pi/Period*300)))*np.sin((2*np.pi/Period*300))
x_H_Q = 3.1*(1. - 0.678**2)/(1. + 0.678*np.cos((2*np.pi/Period*400)))*np.cos((2*np.pi/Period*400))
y_H_Q = 3.1*(1. - 0.678**2)/(1. + 0.678*np.cos((2*np.pi/Period*400)))*np.sin((2*np.pi/Period*400))
x_H_X = 3.1*(1. - 0.678**2)/(1. + 0.678*np.cos((2*np.pi/Period*450)))*np.cos((2*np.pi/Period*450))
y_H_X = 3.1*(1. - 0.678**2)/(1. + 0.678*np.cos((2*np.pi/Period*450)))*np.sin((2*np.pi/Period*450))
x_H_D = 3.1*(1. - 0.678**2)/(1. + 0.678*np.cos((2*np.pi/Period*500)))*np.cos((2*np.pi/Period*500))
y_H_D = 3.1*(1. - 0.678**2)/(1. + 0.678*np.cos((2*np.pi/Period*500)))*np.sin((2*np.pi/Period*500))
x_H_W = 3.1*(1. - 0.678**2)/(1. + 0.678*np.cos((2*np.pi/Period*550)))*np.cos((2*np.pi/Period*550))
y_H_W = 3.1*(1. - 0.678**2)/(1. + 0.678*np.cos((2*np.pi/Period*550)))*np.sin((2*np.pi/Period*550))
x_H_R = 3.1*(1. - 0.678**2)/(1. + 0.678*np.cos((2*np.pi/Period*600)))*np.cos((2*np.pi/Period*600))
y_H_R = 3.1*(1. - 0.678**2)/(1. + 0.678*np.cos((2*np.pi/Period*600)))*np.sin((2*np.pi/Period*600))
x_H_V = 3.1*(1. - 0.678**2)/(1. + 0.678*np.cos((2*np.pi/Period*650)))*np.cos((2*np.pi/Period*650))
y_H_V = 3.1*(1. - 0.678**2)/(1. + 0.678*np.cos((2*np.pi/Period*650)))*np.sin((2*np.pi/Period*650))
```

```
x_H_M = 3.1*(1. - 0.678**2)/(1. + 0.678*np.cos((2*np.pi/Period*700)))*np.cos((2*np.pi/Period*700))
y_H_M = 3.1*(1. - 0.678**2)/(1. + 0.678*np.cos((2*np.pi/Period*700)))*np.sin((2*np.pi/Period*700))
x_H_U = 3.1*(1. - 0.678**2)/(1. + 0.678*np.cos((2*np.pi/Period*750)))*np.cos((2*np.pi/Period*750))
y_H_U = 3.1*(1. - 0.678**2)/(1. + 0.678*np.cos((2*np.pi/Period*750)))*np.sin((2*np.pi/Period*750))
x_H_P = 3.1*(1. - 0.678**2)/(1. + 0.678*np.cos((2*np.pi/Period*800)))*np.cos((2*np.pi/Period*800))
y_H_P = 3.1*(1. - 0.678**2)/(1. + 0.678*np.cos((2*np.pi/Period*800)))*np.sin((2*np.pi/Period*800))
x_H_T = 3.1*(1. - 0.678**2)/(1. + 0.678*np.cos((2*np.pi/Period*850)))*np.cos((2*np.pi/Period*850))
y_H_T = 3.1*(1. - 0.678**2)/(1. + 0.678*np.cos((2*np.pi/Period*850)))*np.sin((2*np.pi/Period*850))
x_H_N = 3.1*(1. - 0.678**2)/(1. + 0.678*np.cos((2*np.pi/Period*900)))*np.cos((2*np.pi/Period*900))
y_H_N = 3.1*(1. - 0.678**2)/(1. + 0.678*np.cos((2*np.pi/Period*900)))*np.sin((2*np.pi/Period*900))
x_H_S = 3.1*(1. - 0.678**2)/(1. + 0.678*np.cos((2*np.pi/Period*950)))*np.cos((2*np.pi/Period*950))
y_H_S = 3.1*(1. - 0.678**2)/(1. + 0.678*np.cos((2*np.pi/Period*950)))*np.sin((2*np.pi/Period*950))
x_H_O = 3.1*(1. - 0.678**2)/(1. + 0.678*np.cos((2*np.pi/Period*1000)))*np.cos((2*np.pi/Period*1000))
y_H_O = 3.1*(1. - 0.678**2)/(1. + 0.678*np.cos((2*np.pi/Period*1000)))*np.sin((2*np.pi/Period*1000))

fig, ax = plt.subplots(figsize=(6,6))

ax.plot(0,0,color='orange',marker='o',linestyle='',markersize=10,markerfacecolor='yellow',label='Su
n')
ax.plot([],[],color='blue',linestyle='dashed',marker='o',label='Earth')
ax.plot([],[],color='brown',linestyle='dashed',marker='o',label='Jupiter')
ax.plot([],[],color='red',linestyle='dotted',marker='o',label='Spacecraft')

ax.plot(-x_H_B,-y_H_B,color='dimgray',marker
='p',markerfacecolor='dimgray',linestyle='',label='Orbital Path')
ax.plot(-x_H_C,-y_H_C,color='dimgray',marker ='p',markerfacecolor='dimgray')
ax.plot(-x_H_A,-y_H_A,color='dimgray',marker ='p',markerfacecolor='dimgray')
ax.plot(-x_H_D,-y_H_D,color='dimgray',marker ='p',markerfacecolor='dimgray')
ax.plot(-x_H_M,-y_H_M,color='dimgray',marker ='p',markerfacecolor='dimgray')
ax.plot(-x_H_N,-y_H_N,color='dimgray',marker ='p',markerfacecolor='dimgray')
ax.plot(-x_H_O,-y_H_O,color='dimgray',marker ='p',markerfacecolor='dimgray')
ax.plot(-x_H_P,-y_H_P,color='dimgray',marker ='p',markerfacecolor='dimgray')
ax.plot(-x_H_Q,-y_H_Q,color='dimgray',marker ='p',markerfacecolor='dimgray')
ax.plot(-x_H_R,-y_H_R,color='dimgray',marker ='p',markerfacecolor='dimgray')
ax.plot(-x_H_S,-y_H_S,color='dimgray',marker ='p',markerfacecolor='dimgray')
ax.plot(-x_H_T,-y_H_T,color='dimgray',marker ='p',markerfacecolor='dimgray')
ax.plot(-x_H_U,-y_H_U,color='dimgray',marker ='p',markerfacecolor='dimgray')
```

```
ax.plot(-x_H_V,-y_H_V,color='dimgray',marker ='p',markerfacecolor='dimgray')
ax.plot(-x_H_W,-y_H_W,color='dimgray',marker ='p',markerfacecolor='dimgray')
ax.plot(-x_H_X,-y_H_X,color='dimgray',marker ='p',markerfacecolor='dimgray')


ax.add_patch(Earth)
ax.add_patch(Jupiter)

#ax.set_xlabel('X [AU]',fontsize=12)
#ax.set_ylabel('Y [AU]',fontsize=12)
#ax.legend(loc='best',fontsize=12)
plt.axis('off')

anim = animation.FuncAnimation(fig, animate, init_func=init, frames=260, blit=True, interval=41,)

from IPython.display import HTML
HTML(anim.to_jshtml())
```

Appendix 1 – Shell Script Used – 4. 'Ideal Rocket Equation'.

```
"""Ideal Rocket Equation.ipynb

Automatically generated by Colaboratory.

Original file is located at
    https://colab.research.google.com/drive/13mn6gBBH0-tuEx_JpwoKaE_wyUm_wZz1
"""

import matplotlib
import matplotlib.pyplot as plt
import math

# Hydrogen based Rocket - Saturn V ;
# Exhaust velocity = 2.4 km/s, Initial mass at launch = 1.19e5kg, Dry Mass empty on fuel = 1e4kg.

v_e = 2400                          # Exhaust Velocity
def delta_velocity(mass_i,mass_f):        # Delta Velocity
    d_v = v_e*math.log(mass_i/mass_f)      # Ideal Rocket Equation
    return d_v

# d_v for a range of mass_f's & mass ratio

d_m = 10                           # Delta Masses (kg's)
mass_i = 1.19e5                        # Initial Mass = 1.19e5 kg
mass_f = 1.0e4                         # Dry Mass = 1e4 kg
d_v_array =[]                        # Delta Velocity Array
mass_i_array =[]                       # Intial Mass Array
mass_f_array = []                      # Final Mass Array
```

```
mass_ratio_array =[]                    # Mass Ratio Array
mass_i_array.append(mass_i)
v_tot = []                              # Total Velocity


while (mass_f<=1.19e5) :
   mass_f_array.append(mass_f)
   mass_ratio_array.append((mass_i/mass_f))
   d_v_array.append(delta_velocity(mass_i, mass_f))
   mass_f+=d_m
   v_tot.append(d_v_array[-1]/v_e)

mass_f_array.reverse()

fig = plt.figure()
# Plotting delta_v vs mass_final
plt.plot(d_v_array, mass_f_array, '-')
plt.rcParams['axes.facecolor'] = 'white'
plt.title("Mass - Velocity Relation (Dry Weight Requirements)")
plt.ylabel("Mass (kg)")
plt.xlabel("Velocity (km/s)")
plt.show()

# Plot delta v vs the mass ratio
plt.plot(d_v_array,mass_ratio_array, '-')
plt.rcParams['axes.facecolor'] = 'white'
plt.title("Mass Ratio - Velocity Relation")
plt.ylabel("Mass Ratio (kg)")
plt.xlabel("Velocity (km/s)")
plt.show()
```

Appendix 2. Risk Assessment

Personal Risk Assessment Listing of All Risks Associated with this project:

Thankfully, the risks associated with undertaking this project are **extremely low**; Rating 1, According to the 'Risk Matrix' outlined in the Laboratory Safety Induction (Credit; Barry Byrne). This is because the potential harm or adverse health effects are minimal. That said, there are some minor risks associated with computer based projects listed below;

- Possible back, neck and shoulder pains due to poor posture when working on computers.

  Control Measure;

Healthy workspace with appropriate furniture to promote good posture [eg; desk chair designed for back support adjusted to correct height at desk] and frequent breaks to stretch muscles and promote blood flow.

- Possible headaches, sleep problems or eyestrain caused by prolonged screen time and blue-light produced in LED screens.

  Control Measure;

  Reduction of screen brightness in display settings, tilting screen to reduce glare on eyes, working in a well-lit room, use of blue-light glasses during screen use.

- Possible joint pain in fingers and hands caused by overuse of keyboard or mouse.

  Control Measure;

  Frequent breaks to stretch and relieve stress on fingers, gently typing on keyboard, keeping wrists flexible when typing rather than at fixed position to avoid repetitive strenuous movements on muscles and ligaments, use of voice assistant typing software [eg; Google Assistant, SpeechTexter].